

Multi-robot consistent formation control based on novel leader-follower model and optimization motion planning approach

Liguo Yao^{a,b,c}, Xiaoyang Yuan^a, Guanghui Li^d, Yao Lu^{a,b,c,*}, Taihua Zhang^{a,b,c,*}

^a School of Mechanical and Electrical Engineering, Guizhou Normal University, Guiyang, Guizhou 550025, China

^b Guizhou Key Laboratory of NewGen Cyberspace Security, Guizhou Normal University, Guiyang, Guizhou 550025, China

^c Technical Engineering Center of Manufacturing Service and Knowledge Engineering, Guizhou Normal University, Guiyang, Guizhou 550025, China

^d School of Automation and Intelligent Manufacturing, Southern University of Science and Technology, Shenzhen, Guangdong 518055, China

ARTICLE INFO

Keywords:

Formation control
Multi-robot consistent formation
Crayfish optimization algorithm
Leader-follower model

ABSTRACT

Path planning and formation control are of great significance for improving the efficiency of robot collaboration. In practical applications, traditional algorithms are complex due to the complexity and immediacy of real scenes. Optimization algorithms with their problem independence and easy scalability, can effectively solve the difficulties encountered in multi-robot path planning and formation control. However, current optimization algorithms generally have problems such as being easily trapped in local optimality and incomplete algorithm exploration. To better solve the problems of significant path trajectory error and poor formation adaptability in the collaborative formation of multi-robot systems. Based on Crayfish Optimization Algorithm (COA), this paper introduces three strategies that effectively improve the optimization effect of the COA, and proposes a hybrid algorithm suitable for collaborative formation of multi-robot systems (SCCOA). We design a novel hierarchical leader-follower formation framework and prove the stability of the formation control model. The proposed SCCOA was combined with the framework to use to adjust and parallel control the trajectory, speed, and other related factors of the leader and follower to realize the formation control system to achieve the goal of multi-robot collaborative formation. To verify the effectiveness of the SCCOA in path planning and formation control systems, SCCOA was compared with 11 excellent algorithms in experiments, and various evaluation indicators were established by analyzing various coefficients in multi-robot path planning and formation control. Results showed that SCCOA ranked first in all metrics, demonstrating the excellent performance of the SCCOA in solving multi-robot path planning and formation control problems.

1. Introduction

Robots are the product of the development of theories and technologies in multiple fields such as information processing, computer engineering, sensor technology, and automatic control technology [1,2]. With the continuous development of emerging technologies such as artificial intelligence and 5 G communications, robots are becoming increasingly intelligent [3]. Mobile robots can move and perform various tasks, and their development history can be traced back to the early 20th century. As a robotic system with autonomous action, self-positioning, perception, and decision-making capabilities [4], mobile robots can operate autonomously in different environments and are widely used in industrial production lines [5], warehouse automation [6], home services, medical care [7], and other fields. They can improve

work efficiency and production benefits, reduce labor costs and production safety hazards, and increase social productivity. In addition, mobile robots can replace humans in exploring, detecting, and rescuing dangerous or harsh environments, while also being able to quickly and efficiently obtain the required information [8]. A multi-robot system is a complex intelligent system comprised of multiple robots, characterized by coordination, control, environmental perception, and information sharing. In multi-robot applications, complex tasks are broken down into several subtasks, each completed collaboratively by the robots. This not only improves overall workload capacity and shortens completion time, but also provides strong adaptability to the environment, enabling coordinated re-planning of tasks in dynamic and complex environments, ensuring system fault tolerance and stability [9]. For example, methods such as limited-time reinforcement learning control for docking

* Corresponding author at: School of Mechanical and Electrical Engineering, Guizhou Normal University, Guiyang, Guizhou 550025, China.

E-mail address: zhangth542@gznu.edu.cn (T. Zhang).

<https://doi.org/10.1016/j.knosys.2025.114590>

Received 22 August 2025; Received in revised form 25 September 2025; Accepted 1 October 2025

Available online 2 October 2025

0950-7051/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

unmanned aerial vehicles (UAVs) and unmanned ground vehicles (UGVs) in complex environments [10]. Bilal et al. proposed an autonomous trajectory tracking system for wheat farms based on quadcopter drones [11].

Although the control procedures and intelligence involved in multi-robot systems have been developed for a long time [12], research on improving the working efficiency of multi-robot systems still needs to be strengthened. Under the requirements of autonomy and unmanned collaboration, the system has high requirements for algorithms involved in intelligent decision-making. In most of its application fields, the ability of autonomous navigation and the collaborative movement ability of the system formation determine the success or failure of such robots in performing tasks and their efficiency [13]. Path planning in complex environments is one of the key issues that needs to be paid attention to in robot autonomous navigation technology, and it is also a technical breakthrough for autonomous navigation [14]. Multi-robot formation control can significantly improve task execution capabilities by coordinating multiple robots to form specific formations and move in coordination, and has become a research hotspot in the field of robotics [15].

With the convergence of robotics, control theory, and artificial intelligence, its theoretical framework has gradually matured and attracted widespread attention in academia and industry both domestically and internationally [16,17]. Multi-robot collaborative formation control involves multiple robots performing tasks such as searching large areas and operating in confined areas through division of labor, collaboration, and intelligent behaviors such as formation transformation. Formation and maintenance of multiple robots involves controlling the postures of multiple robots to form and maintain a desired formation according to specific geometric relationships [18]. Formation transformation primarily involves transitioning a robot formation from its original formation to a new target formation due to changes in the environment or task requirements. Methods involved in formation control are categorized as classical control methods and modern intelligent methods. Classical control methods include leader-follower methods, virtual structure methods, and behavior-based methods. Modern intelligent methods include deep learning, reinforcement learning, distributed optimization control, and swarm intelligence [19]. Among them, the leader-follower method was proposed by Balch and Arkin in the early days [20]. By setting the trajectory of the leader robot, the followers achieve formation based on relative position or distance tracking. Desai and Kumar from the University of Texas team introduced a dynamic leader switching mechanism to avoid system crash caused by a single leader failure [21]. Aslam et al. studied the formation control problem of a class of heterogeneous multi-agent systems with time delays and established a heterogeneous multi-agent system model with input delay and communication delay [22]. Tan et al. proposed a hierarchical safe reinforcement learning with predicted performance control (HSRL-PPC) scheme that can effectively address the challenges encountered by leader-follower systems in complex environments [23]. The virtual structure method was proposed by Lewis and Tan [24]. The formation is regarded as a rigid geometric body, and each robot tracks the reference point of the virtual structure. Ren decentralized the virtual structure in the multi-vehicle system through a consistency strategy and successfully applied the distributed control framework to vehicle formation simulation [25]. Aslam et al. proposed an efficient filter based on H_∞ delay filtering for Markov jump fuzzy systems [26]. Monteiro and Bicho proposed a behavior-based formation control dynamics method [27], which was verified to be effective in a simulation environment. Xue et al. designed a formation tracking control strategy based on a leader-follower framework, which can effectively reduce collision risks and improve motion efficiency [28]. Shamrooz et al. proposed a modeling method for asynchronous pattern-dependent delay of robots based on random Markov jump patterns based on static neural networks [29].

In the field of intelligent learning, Sui et al. applied deep

reinforcement learning to multi-robot formation systems and designed a model-guided method consisting of a formation controller based on consistency theory and an optimal collision avoidance strategy [30]. Tan et al. proposed a robust optimal control method to effectively improve the anti-interference ability of the robot system [31]. Aslam et al. proposed an innovative adaptive tracking method based on reinforcement learning to solve the time-varying delay problem of actuator failure in complex systems [32]. Xiang et al. used multiple strategies to improve the deep Q learning network and constructed a high-fidelity semi-physical simulation system to verify the proposed UAV formation cooperation algorithm [33]. Ru et al. used game theory to design a distributed predictive control algorithm based on Nash negotiation [34]. Through simulation experiments, they verified the algorithm's effectiveness in controlling the autonomous reconstruction, obstacle avoidance, and formation cooperation. Bilal et al. proposed a new IoRT architecture based on transfer learning (TL) technology [35]. Tan et al. designed a finite-time algorithm for the safe control of a quadcopter system using a reinforcement learning method for multi-person non-zero-sum games [36].

In the field of bionic swarm intelligence optimization, Tang et al. proposed a formation control method of a multi-agent PSO algorithm [37], which effectively optimized the relevant parameters of the multi-agent system. They verified the effectiveness of the proposed method in the optimal time control problem of formation reconstruction. Hasan and Raafat applied PSO to consistent multi-agent system control [38], accelerating the convergence of the formation position. Kamel et al. designed an optimization method integrating CPTD, GA, and PSO [39], and proposed a fault-tolerant formation control strategy. The proposed hybrid method can minimize the configuration time and improve the fault tolerance of the UAV system. Noor et al. used the asexual reproduction optimization method to propose a new mobile robot formation controller based on the potential field method [40], suitable for real-time control applications of formation systems. Qiu et al. proposed an improved Brainstorming optimization algorithm and verified the effectiveness and superiority of the algorithm in the nonlinear rolling horizon control mode of unmanned aerial vehicles through simulation experiments [41]. Duan et al. designed a pigeon-inspired optimization and verified the algorithm's effectiveness in aerial robot path planning applications through simulation experiments [42]. Zhang et al. proposed a new predator-prey pigeon flock optimization algorithm [43], which significantly improved the adaptability of three-dimensional path planning for unmanned combat aircraft. Wu et al. proposed a global-local moth flame optimization algorithm [44], which has the advantages of speed and optimality in the path planning problem of unmanned aerial vehicles.

In summary, research on formation control technology focuses on improving classical methods and hybridizing classical methods with intelligent learning algorithms. By designing fitness functions and diverse path manipulation strategies [45], the algorithm's convergence speed and solution quality can be improved, adapting to environmental obstacles and dynamic changes, supporting complex multi-objective, multi-constrained optimization tasks, and enhancing the system's robustness and fault tolerance. This allows for more efficient planning of conflict-free paths and reasonable formation motion among multiple robots, achieving stable and coordinated collaborative control [46]. This makes algorithms particularly suitable for solving high-dimensional, strongly coupled, and dynamically changing path planning and formation control problems in multi-robot systems. However, existing research on metaheuristic-based formation control methods has largely focused on relatively simple algorithms such as GAs and PSOs, with relatively little application to complex formation control problems [47]. Therefore, to better solve the complex formation control problem in multi-robot collaborative formations, this paper adds the exploration and exploitation process and mutation operator of the sine and cosine algorithms to the COA algorithm, and balances the exploration and exploitation weight ratio. The improved algorithm is called SCCOA,

which has better path planning accuracy and processing efficiency in multi-robot path planning and formation control. To verify the effectiveness of SCCOA, SCCOA was compared with 11 algorithms (COA [48], PSO [49], HHO [50], SCA [51], ACO [52], WOA [53], GJO [54], BWO [55], AGWO [56], MPSO [57] and TACPSO [58]) on path planning and formation control problems. The SCCOA algorithm significantly improves the ability to plan multi-robot paths and control formation. The main summary contributions of this paper are as follows.

- Aiming at the problems encountered in multi-robot path planning and formation control, a hybrid improved optimization algorithm (SCCOA) suitable for multi-robot system path planning and formation control is proposed to further improve the accuracy and stability of multi-robot path planning and formation control systems.
- By analyzing the problems encountered in multi-robot path planning and formation control, a hierarchical leader-follower formation architecture was designed, and the stability of the formation control model was proved.
- The various coefficients in multi-robot path planning and formation control were analyzed, multiple evaluation indicators were established, and the speed and position errors of the formation system were analyzed.
- The proposed algorithm SCCOA is compared with 11 excellent algorithms including COA in experiments to further demonstrate the excellent performance of SCCOA in multi-robot path planning and formation control systems.

The remainder of this paper consists of the following: Section 2 presents the prerequisites and problem definition. Section 3 introduces the core control and planning algorithms for formation control. Section 4 describes the formation control framework and the stability evaluation and performance evaluation metrics of the proposed control model. Section 5 conducts simulation experiments on various algorithms for formation control and compares and analyzes the experimental results. Section 6 concludes with a summary and plans for future work. All the symbols and variables in this paper are shown in Table 1.

2. Multi-robot formation control problem

2.1. Agent dynamics model

The agent dynamics model primarily divides the entire formation system into a leader and a follower. The leader determines the overall

direction and speed of the formation, while the followers autonomously adjust their positions and speeds based on their relationships with other agents to maintain formation. The kinematic model of each agent (leader and follower) is converted into discrete time form, which can be described by Eq. (1).

$$\begin{cases} p_i(k+1) = p_i(k) + v_i(k)\Delta t \\ v_i(k+1) = v_i(k) + u_i(k)\Delta t \end{cases} \quad (i = 1, \dots, N)$$

$$p_i(k) = [x_i(k), y_i(k)]^T \in \mathbb{R}^2,$$

$$v_i(k) = [v_{x_i}(k), v_{y_i}(k)]^T \in \mathbb{R}^2,$$

$$u_i(k) \in \mathbb{R}^2$$

where, $p_i(k)$ represents the position at time k , $v_i(k)$ represents the velocity at time k , $u_i(k)$ represents the control input (acceleration), and Δt represents the discrete time step.

The expected deviation of the follower relative to the leader can be described by Eq. (2).

$$\delta_i = [\delta_{x_i}, \delta_{y_i}]^T \quad (2)$$

where, the deviation δ_i is expressed as a coordinate matrix and can be set in advance.

The actual formation error $e_i(k)$ is defined as Eq. (3).

$$e_i(k) = p_L(k) + \delta_i - p_i(k) \quad (3)$$

where, $p_L(k)$ is the position of the leader and $p_i(k)$ is the position of a follower.

2.2. Formation control target model

Multi-robot formation control involves the coordinated motion of multiple robots in a dynamic or static environment to form and maintain a specific geometric configuration while simultaneously meeting obstacle avoidance, communication constraints, and motion performance requirements. Its core objective is maintaining a stable relative positional relationship between the robots while completing a global task (e.g., reaching a target point) through distributed or centralized control strategies [59]. The control objectives of a wheeled robot pilot-follower consistency control method are detailed below, using the example of this method.

The control objectives of the multi-robot formation control problem are divided into formation maintenance, obstacle avoidance re-

Table 1
Summary of symbols and variables used in this paper.

Symbol	Interpretation	Symbol	Interpretation	Symbol	Interpretation
$P_i(k)$	Position at time k	$v_i(k)$	Speed at time k	$u_i(k)$	Control input (acceleration)
Δt	Discrete time step	δ_i	Coordinate matrix	$p_L(k)$	Navigators position
r_j	Obstacle radius	x_L, y_L	Navigators Coordinates	ϵ_x, ϵ_y	Tolerance threshold
r_{obs}^j	Obstacle radius	d_{safe}	Minimum safety distance	x_L^T	Position of the pilot at termination
x_{goal}	End point coordinate position	ς	Accuracy threshold	v_{max}	Maximum movement speed
$\mathcal{O}_{static}$	Predefined obstacle information matrix	$A_{ij}(t)$	Dynamic adjacency matrix	α, β, γ	Weight coefficient
ϕ	Obstacle potential field function	p_i	The follower's position vector	p_L	Position vector of the leader
o_j	The obstacle's position vector	g	Target location	CF	The location of the initial solution
N	Population size	dim	Population dimension	$f_{obs,i}$	Potential field function
ub_i, lb_j	Upper and lower limits	$rand_k$	A random number between 0 and 1)	$temp$	Ambient temperature parameters
CF_G	Optimal solution location	$f_{obs,i}$	Potential field function	t	Current iteration number
M	Number of obstacles	round	Rounding function	T	Maximum number of iterations
$fitness_i$	Function value of the i -th agent	$fitness_{food}$	Fitness function value of food location	Gm	Gaussian mutation operator
I_c	Moment of inertia of the population	Chs	Chaos coefficient	θ	Adjustable coefficient
v_L^{opt}	Navigators Speed	$Explr$	Explore proportions	$Explr$	Development ratio
$X_{i,j}$	The j -th dimension position of the i -th individual	c_i	The central position of the i -th individual	$L_{i,j}$	The position of the i -th agent in the population in the j -th dimension
$\otimes$	Field speed integration	K_f	Error tracking gain	K_g	Navigation gain
R_c	Communication radius	N_F	Number of followers	Δ_i	Expectation bias

quirements, target convergence, motion performance, etc. Its mathematical model can be expressed as Eq. (4).

$$\begin{aligned}
 & \underset{v_i(t)}{\text{minimize}} \quad \sum_{i=1}^N (\alpha \|v_i - v_{ref}\|^2 + \beta \|x_i - (x_L + \delta_i)\|^2) \\
 & \text{subject to} \quad \text{Kinematic model:} \quad \dot{x}_i = v_i \\
 & \quad \text{Obstacle avoidance constraints:} \quad \|x_i - x_{obs}^j\| \geq r_{obs}^j + d_{safe} \\
 & \quad \text{Communication connectivity:} \quad \text{rank}(A(t)) = N - 1 \\
 & \quad \text{Velocity constraints:} \quad \|v_i\| \leq v_{max}
 \end{aligned} \quad (4)$$

where, each constraint will be described in detail.

(1) Formation maintenance

The goal of formation maintenance is reflected in the geometric constraints, requiring the relative distance between individuals in the formation to remain within the set formation relative position range. Taking the leader-follower method as an example, its formation maintenance goal can be described as Eq. (5).

$$\begin{aligned}
 & \lim_{t \rightarrow \infty} \|x_i(t) - (x_L(t) + \delta_x^i)\| \leq \epsilon_x \\
 & \lim_{t \rightarrow \infty} \|y_i(t) - (y_L(t) + \delta_y^i)\| \leq \epsilon_y
 \end{aligned} \quad (5)$$

where, x_L and y_L are the navigator coordinates, ϵ_x and ϵ_y are the allowable error thresholds.

(2) Obstacle avoidance constraints

The obstacle avoidance constraint is an inequality constraint, which requires that the relative distance between the robot and the obstacle is not less than the set safety distance. Its mathematical expression is Eq. (6).

$$\min_j (\|x_i - x_{obs}^j\| - r_{obs}^j) \geq d_{safe} \quad (6)$$

where, r_{obs}^j is the radius of the obstacle and d_{safe} is the minimum safe distance.

(3) Target Convergence

Target convergence is the leader's final state constraint, which requires the formation to eventually converge to the terminal position. The mathematical expression is Eq. (7).

$$\|x_L^T - x_{goal}\| \leq \varsigma \quad (7)$$

where, x_L^T is the position of the navigator at the end, x_{goal} is the coordinate position of the end point, and ς is the accuracy threshold.

(4) Sports performance

The control objective of motion performance is expressed as a speed constraint, which requires that certain state quantities of all members of the formation be bounded. The mathematical expression is Eq. (8).

$$\|v_i(t)\| \leq v_{max} \quad (8)$$

where, v_{max} is the maximum movement speed.

(5) Environmental constraints

Environmental constraints are divided into obstacle constraints and communication constraints, which are set according to actual needs and simulation requirements. Obstacle constraints can be expressed as Eq. (9), and communication constraints can be expressed as Eq. (10).

$$\mathcal{O}_{static} = \left\{ (x_{obs}^j, y_{obs}^j, r_{obs}^j) \right\}_{j=1}^M \quad (9)$$

$$A_{ij}(t) = 1 \Leftrightarrow \|x_i(t) - x_j(t)\| \leq R_{com} \quad (10)$$

where, $\mathcal{O}_{static}$ is the predefined obstacle information matrix, which contains the location information and radius of the obstacle; $A_{ij}(t)$ is the dynamic adjacency matrix, which represents the dynamic changes within the limited communication range R_{com} .

The multi-agent formation control problem is a multi-objective

collaborative optimization problem that must simultaneously satisfy the three major constraints of formation maintenance, dynamic obstacle avoidance, and global navigation. The mathematical objective function of this problem is Eq. (11).

$$\min_{v_i} \sum_{i=1}^N \left(\alpha \|p_i - (p_L + \Delta_i)\|^2 + \beta \sum_{j=1}^M \phi(p_i, o_j) \right) + \gamma \|p_L - g\|^2 \quad (11)$$

where, α , β , and γ are weight coefficients, ϕ is the obstacle potential field function, p_i is the follower's position vector, p_L is the leader's position vector, o_j is the obstacle's position vector, and g is the target position. The entire control process must be implemented in a distributed optimization under the communication constraint $A_{ij}(t)$.

3. Formation control optimization method

3.1. Crayfish optimization algorithm (COA)

COA is a new meta-heuristic algorithm developed by simulating crayfish's biological behavior. Although COA has been validated on several benchmark functions and engineering optimization problems, the population diversity and search range in the algorithm's initial stages need to be improved, leading to its limited effectiveness on some problems. This is especially true for more complex hybrid functions, which can easily fall into local optima in the later stages of iteration.

3.1.1. Initialize the population

In the COA algorithm, each individual crayfish serves as a search agent. For a multidimensional optimization problem, each agent is a matrix with 1 row and dim columns, where dim represents the dimension of the problem. The form of the solution is determined by the total number of individuals in the population (N) and the total dimension of the search space (dim), as expressed in Eq. (12). After defining the problem dimension and the upper and lower bounds (ub , lb) of the solution space, the COA generates an initial solution set in a random manner. Its mathematical expression is shown in Eq. (13).

$$CF = [CF_1, CF_2, CF_3, \dots, CF_N] = \begin{bmatrix} CF_{1,1} & \dots & CF_{1,j} & \dots & CF_{1,dim} \\ \dots & \dots & \dots & \dots & \dots \\ CF_{i,1} & \dots & CF_{i,j} & \dots & CF_{i,dim} \\ \dots & \dots & \dots & \dots & \dots \\ CF_{N,1} & \dots & CF_{N,j} & \dots & CF_{N,dim} \end{bmatrix} \quad (12)$$

$$CF_{i,j} = ub_j + (ub_j - lb_j) \times rand_1 \quad (13)$$

where, CF represents the individual position after initialization, N represents the total number of individuals in the population of the algorithm, dim represents the number of dimensions in the algorithm, $CF_{i,j}$ is the position of the i -th agent in the j -th dimension, ub_j and lb_j represent the upper and lower bounds of the algorithm in the j -th dimension, respectively, and $rand_k$ ($k=1, 2, \dots, n$) represents a random number between 0 and 1 (excluding 1).

3.1.2. Parameter settings

Environmental temperature and food are important factors affecting crayfish growth and development. The algorithm defines two control factors: temperature $temp$ and food intake rate p , which influence individual crayfish behavior. $temp$ is calculated using Eq. (14), and p is calculated using Eq. (15).

$$temp = rand_2 \times 15 + 20 \quad (14)$$

$$p = C_1 \times \left(\frac{1}{\sqrt{2 \times \pi \times \sigma}} \times \exp\left(-\frac{(temp - \mu)^2}{2 \times \sigma^2}\right) \right) \quad (15)$$

where, μ represents the suitable temperature for crayfish to forage, and its value is a constant of 25. σ and C_1 are used to control the feeding rate

of crayfish at different temperatures, and their values are 3 and 0.2, respectively.

3.1.3. Algorithmic bionic behavior (summer escape and competition phases)

When $temp > 30$, the crayfish will dig a burrow to avoid the heat. The burrow location CF_{shade} is defined as follows:

$$CF_{shade} = (CF_G + CF_L) / 2 \quad (16)$$

where, CF_G represents the current optimal search individual position, and CF_L represents the optimal position of the population individual.

The competition phase results from the crayfish's scrambling for burrows, while the summer escape phase results from their entering burrows to escape the heat. The COA presents the competition and summer escape phases as random events.

When $temp > 30$ and $rand < 0.5$, the algorithm enters the summer vacation phase, and the individual position changes using Eq. (17).

$$CF_{ij}^{t+1} = CF_{ij}^t + C_2 \times rand_3 \times (CF_{shade} - CF_{ij}^t) \quad (17)$$

where, t represents the number of times the algorithm is run at this stage, and $t+1$ represents the number of subsequent runs, CF_{ij}^t represents the position of the i -th agent in the j -th dimension in the current iteration, CF_{ij}^{t+1} represents the position of the i -th agent in the j -th dimension in the next iteration, and C_2 is defined by Eq. (18).

$$C_2 = 2 - (t / T) \quad (18)$$

where, T represents the maximum number of iterations during the algorithm search process.

When $temp > 30$ and $rand > 0.5$, the algorithm enters the competition phase and the agent's position is updated using Eq. (19).

$$CF_{ij}^{t+1} = CF_{ij}^t - CF_{zj}^t + CF_{shade} \quad (19)$$

where, CF_{zj}^t represents a random agent position during the t -th iteration, and z is defined by Eq. (20).

$$z = \text{round}(rand_4 \times (N - 1)) + 1 \quad (20)$$

where, round represents the rounding function.

When $temp \leq 30$, crayfish will leave their burrows and forage for food in order to grow and develop, as reflected in the foraging phase. After the crayfish finds food, if the food is too large, it will tear it apart with its claws and eat it. The location of food that crayfish search for is defined as in Eq. (21).

$$CF_{food} = CF_G \quad (21)$$

The state Q of food is defined as Eq. (22).

$$Q = C_3 \times rand_5 \times (fitness_i / fitness_{food}) \quad (22)$$

where, C_3 represents the maximum circumference of the food, which is a constant 3, $fitness_i$ represents the fitness function value of the i -th agent, and $fitness_{food}$ represents the fitness function value of the food location.

When $temp \leq 30$ and $Q > (C_3 + 1) / 2$, the crayfish will tear the food into pieces with its claws, and the position of the food is updated using Eq. (23). The position of the agent is updated using Eq. (24).

$$CF_{food} = CF_{food} \times e^{(-1/Q)} \quad (23)$$

$$CF_{ij}^{t+1} = CF_{ij}^t + CF_{food} \times p \times (\cos(2 \times \pi \times rand_6) - \sin(2 \times \pi \times rand_7)) \quad (24)$$

where, all variables in the Equation have been defined above.

When $temp \leq 30$ and $Q \leq (C_3 + 1) / 2$, the crayfish eats directly and the agent position is updated using Eq. (25).

$$CF_{ij}^{t+1} = ((CF_{ij}^t - CF_{food}) + (CF_{ij}^t \times rand_8)) \times p \quad (25)$$

During the foraging phase, the crayfish's position update depends on the feeding rate p and the size Q of the food.

The process of the COA algorithm is introduced in the form of pseudo code as shown in Algorithm 1:

3.2. Sin-cosine crayfish optimization algorithm (SCCOA)

Population-based algorithms often have inherent limitations due to the source of their heuristic information, and each algorithm has its most applicable scope [60]. Taking COA as an example, its algorithm has two exploitation strategies and only one exploration strategy. In addition, the parameter $temp$ that controls the exploration-exploitation balance in COA is not flexible. It focuses too much on algorithm exploitation, resulting in a limited search range and a tendency to fall into local optimality. In addition, the scaling range of the agent position movement step size is less diverse, which affects the parallelism and flexibility of the algorithm for complex problems.

The original COA focused on the crayfish's foraging process, specifically its exploitation process, which helped the algorithm converge quickly and possessed a certain ability to find the optimal solution. Analysis revealed that COA converges quickly on simple problems, particularly for simple, single-modal functions, where it performs well. However, as the complexity of the problem increases, COA's optimization capabilities gradually weaken. For complex functions, COA's limitations include insufficient search range, reduced population diversity, susceptibility to local optima, and difficulty escaping from local optima. Furthermore, careful observation of the behavior of crayfish in nature and their living environments revealed that the original COA lacked the ability to simulate the adaptive behavior of crayfish in environments with wider temperature ranges. Furthermore, formation control requires algorithms with high path search probability, completeness, and position change adaptability.

Therefore, this paper makes several improvements to the original COA to make it more suitable for formation control problems. First, it introduces the exploration and exploitation process of the sine-cosine algorithm to mathematically address the theoretical deficiencies of the algorithmic framework and enhance the algorithm's search probability, completeness, and population diversity. Second, it balances the exploration and exploitation weight ratio (competitive process) to prevent excessive algorithm exploitation and reduce the probability of the algorithm falling into a local optimum. Finally, it introduces a mutation operator to eliminate suboptimal solutions and further improve the quality of the current solution. This improved algorithm is called SCCOA.

3.2.1. Sin-Cosine Algorithm

The Sin-Cosine Algorithm is a metaheuristic algorithm based on the sine and cosine laws proposed by Mirjalili [61]. It has unique sine and cosine search rules, which can present a more comprehensive search behavior throughout the iterative process. Its core mathematical model is Eqs. (26) and (27).

$$X_{ij}^{t+1} = X_{ij}^t + C_2 \times \sin(2 \times \pi \times rand_9) \times |2 \times X_{best} - X_{ij}^t| \quad (26)$$

$$X_{ij}^{t+1} = X_{ij}^t + C_2 \times \cos(2 \times \pi \times rand_{10}) \times |2 \times X_{best} - X_{ij}^t| \quad (27)$$

where, X_{ij}^t and X_{ij}^{t+1} correspond to CF_{ij}^t and CF_{ij}^{t+1} above, respectively, and X_{best} corresponds to CF_{food} above.

3.2.2. Balanced exploration and exploitation weight ratio

Through observation of the living environment of crayfish, it was found that they have a wide adaptability to the actual ambient tem-

Algorithm 1

Crayfish Optimization Algorithm (COA).

Input: maximum number of iterations T , population size N , variable dimension dim , fitness function $fit_function$
Output: location information and fitness value of the optimal individual
 Randomly generate initial solutions
 Calculate the fitness value to obtain the current global optimal solution (f_{now}) and the local optimal solution CF_G , CF_L
While $t < T$
 The value of parameter $temp$ is calculated by Eq. (14)
If $temp > 30$
 Define CF_{shade} by Eq. (16)
If $rand < 0.5$
 Update the individual position of the agent (crayfish) using Eq. (17)
Else
 Update the agent's individual position using Eq. (19)
End
Else
 Calculate the values of p and Q using Eqs. (14) and (22)
If $Q > 2$
 Update the food position by Eq. (23)
 Update the agent position by Eq. (24)
Else
 Update the agent position by Eq. (25)
End
End
 Update the fitness value of the population, update CF_G , CF_L
 $t = t + 1$
End

perature. Therefore, the balance parameter $temp$ in the algorithm was modified to balance the weight ratio of exploration and exploitation. The redefined ambient temperature parameter is defined by Eq. (28).

$$temp = (rand_{11} - \beta) \times 25 + 20 \quad (28)$$

where, β represents the geographical location and climate stability of the individual's environment, and is taken as a constant here with a value of 0.2.

However, since the exploitation proportion of the original algorithm was too large, the competition stage was considered to be removed, so the algorithm's bionic behaviors only left the summer escape and foraging behaviors.

At the same time, when $temp > 20$, the algorithm enters the foraging stage, and the exploitation process in COA is replaced by Eq. (29).

$$CF_{ij}^{t+1} = CF_{ij}^t + K_2 \times p \times Q \times (K_3 \times CF_{food} - CF_{ij}^t) \quad (29)$$

where, p and Q are defined by Eqs. (12) and (19), respectively, and K_3 is defined by Eq. (30).

$$K_3 = K_2 \times \left(\exp\left(\frac{t-T}{T}\right) \right)^h \quad (30)$$

where, h is a constant 8, and K_2 is defined by Eq. (31).

$$K_2 = 2 - 2 \times \sin\left(0.5 \times \pi \times \left(\frac{t}{T}\right)^w\right) \quad (31)$$

where, w is a constant whose value is 4, and t and T have the same meaning as above.

3.2.3. Gaussian mutation operator

In the later stages of the algorithm's spatial search, COA is prone to falling into local optima for challenging mixed-dimensional problems. To address this issue, the Gaussian mutation operator (Gm) is introduced to further enhance the algorithm's ability to escape from local optima [62]. Its application in the algorithm is expressed as Eq. (32).

$$CF_i^t = CF_i^t \times (1 + Chs \times Gm), \quad i = 1, 2, 3 \dots N \quad (32)$$

where, Gm is the Gaussian mutation operator and Chs is the chaos coefficient, which are defined by Eqs. (33) and (34).

$$Chs = \vartheta \times (\sin(2 \times \pi \times rand_{13}))^2 + (\cos(2 \times \pi \times rand_{14}))^2 \quad (33)$$

$$Gm = Gs - 0.5 \quad (34)$$

where, ϑ is the adaptive mutation coefficient. The value of this coefficient increases with the number of iterations. Its value range is (0, 1), and Gs is a random number that obeys Gaussian distribution.

The pseudo code of the SCCOA algorithm as shown in Algorithm 2: The flow chart is shown in Fig. 1:

3.3. Exploration and exploitation behavior analysis

Analysis of an optimization algorithm's exploration and exploitation behavior can comprehensively reflect the algorithm's optimization performance [63]. A larger proportion of the exploration phase indicates that the algorithm demonstrates greater ability to search for the optimal solution region and can broadly explore the solution space. A higher proportion of the exploitation phase indicates that the algorithm's agent is more capable of continuously refining and evolving toward the optimal solution location within the current region. The balance between exploration and exploitation has always been a key factor influencing the overall performance of an algorithm. Properly adjusting the ratio of the two helps the algorithm avoid falling into local optimality while ensuring convergence speed, thereby more effectively finding the global optimal solution. By balancing exploration and exploitation, the optimization algorithm can balance the breadth of global search with the precision of local optimization, achieving better search results and improved performance. Its mathematical model is shown in Eq. (35).

$$Explt\% = \frac{Div}{Div_{max}} \times 100, \quad Explr\% = \frac{|Div - Div_{max}|}{Div_{max}} \times 100, \quad (35)$$

$$Div_j = \frac{1}{N} \sum_{i=1}^N \text{median}(X_j) - X_{ij}, \quad Div = \frac{1}{D} \sum_{j=1}^D Div_j$$

where, $Explt$ represents the exploration ratio, $Explr$ represents the exploitation ratio, X_{ij} represents the position of the i -th individual in the j -th dimension, $\text{median}(X_j)$ represents the median of all individuals in the j -th dimension, and N represents the population size. Fig. 2 shows the exploration and exploitation ratio of the original algorithm and SCCOA on the high-dimensional CEC2019 test function set is shown. It can be

Algorithm 2

Sin-Cosine Crayfish Optimization Algorithm (SCCOA).

Input: maximum number of iterations T , population size N , variable dimension dim , fitness function $fit_function$
Output: location information and fitness value of the optimal individual
 Randomly generate initial solutions
 Calculate the fitness value to obtain the current global optimal solution (f_{now}) and the local optimal solution CF_G , CF_L .
While $t < T$
 The value of parameter $temp$ is calculated by Eq. (28)
If $temp > 30$
 Define CF_{shade} by Eq. (16)
 Update the agent's individual position using Eq. (17)
Else If $temp > 20$
 Calculate the values of p and Q using Eqs. (14) and (22)
If $Q > 2$
 Update the food position by Eq. (23)
 Update the agent position by Eq. (24)
Else
 Update the agent position by Eq. (30)
End
Else
If $rand > 0.5$
 Update the food position by Eq. (26)
Else
 Update the agent position by Eq. (27)
End
If $f_{now} < f_{avg}$
 Update the agent position by Eq. (32)
End
 Update the fitness value of the population, update CF_G , CF_L
 $t = t + 1$
End

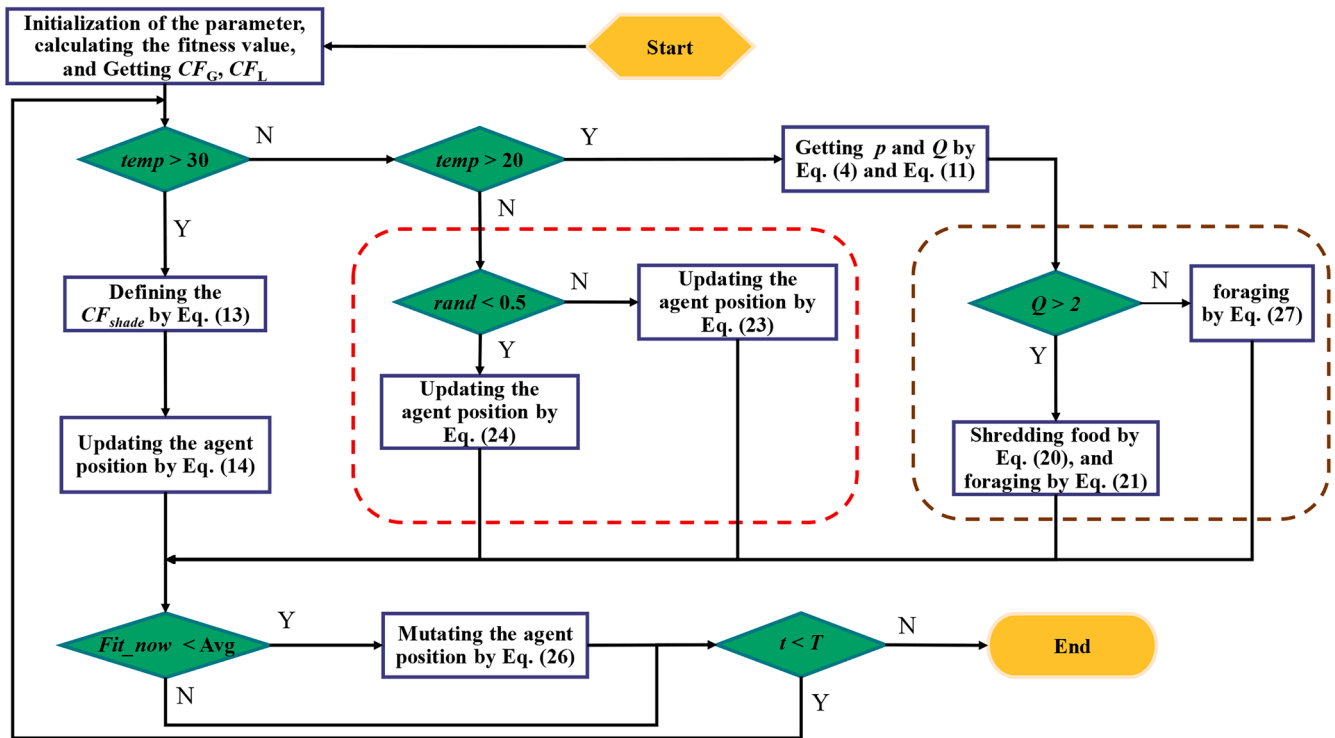


Fig. 1. Flowchart of the Sin-Cosine Crayfish Optimization Algorithm (SCCOA).

seen from the figure that compared with the original algorithm, SCCOA can achieve the best balance between exploration and exploitation in high-dimensional space.

3.4. Population diversity analysis

Population diversity helps measure an algorithm's ability to avoid

local optimality. In population-based metaheuristic algorithms, maintaining a high population diversity can significantly enhance the algorithm's ability to escape the local optimal trap and find the global optimal solution. A selection of test functions was used to measure the population diversity of the proposed algorithm compared to the original algorithm [64]. The moment of inertia was also used to measure population diversity, and its mathematical model is as follows:

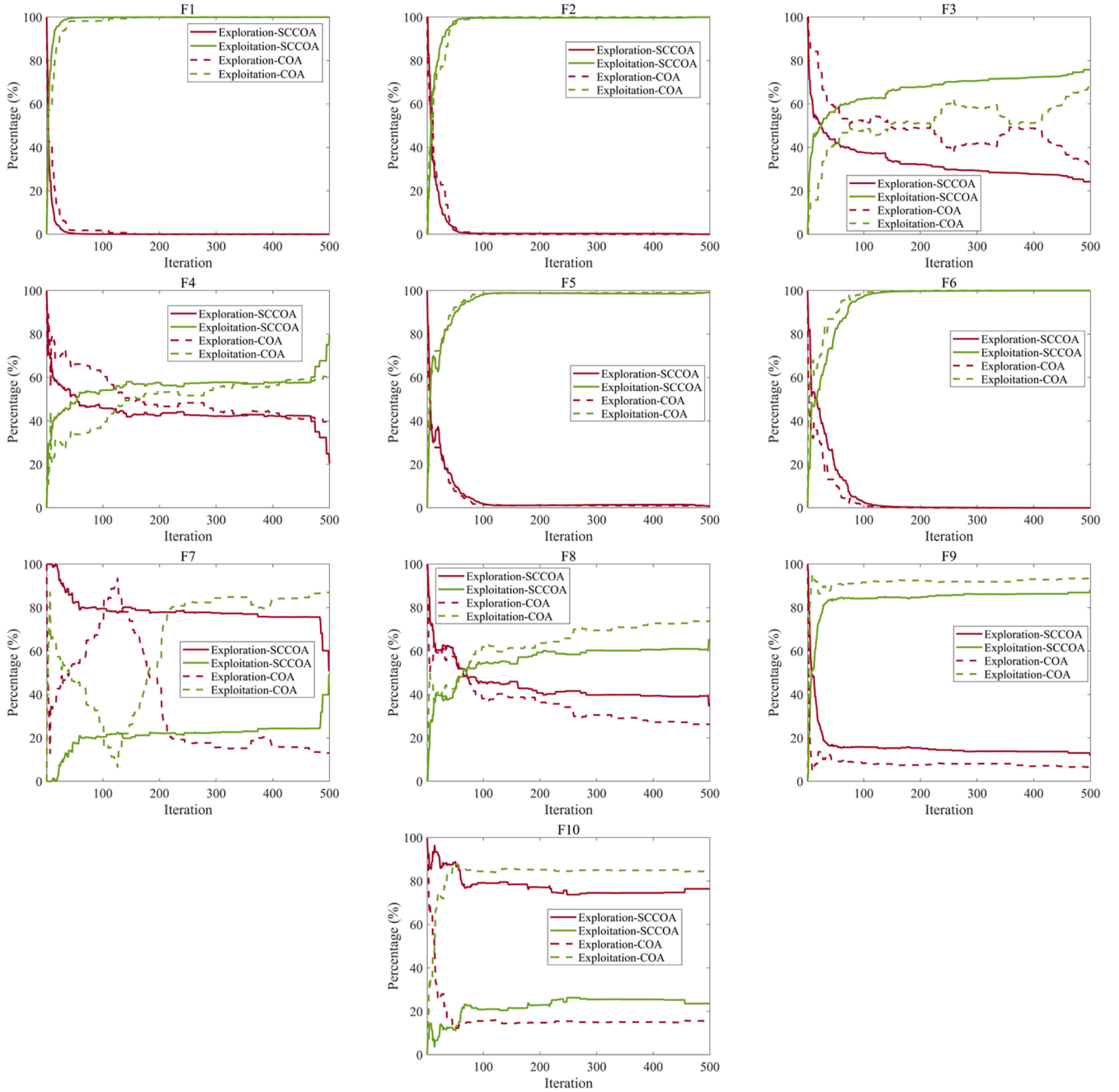


Fig. 2. COA and SCCOA exploration and exploitation ratios.

$$I_c = \sum_{i=1}^N \sum_{j=1}^D (L_{ij} - c_i)^2, \quad c_i = \frac{1}{N} \sum_{j=1}^D L_{ij} \quad (36)$$

where, I_c represents the population's dispersion and the mass center in each iteration, L_{ij} represents the value of the j -th dimension of the i -th individual. Fig. 3 shows a comparison of population diversity comparison between the original algorithm and SCCOA on some high-dimensional CEC2014 test functions is shown. As can be seen from the Fig. 3, in almost all test functions, the population diversity of SCCOA is always greater than that of the original algorithm, further demonstrating that SCCOA's optimization ability is superior to that of the original algorithm.

3.5. Algorithm time complexity

The running speed of an algorithm is generally evaluated by analyzing its time complexity. The parameters involved include the initial population size N , the function's evaluation loss C , the number of algorithms run T , and the spatial dimension D of the searched individuals. Time complexity is denoted by the letter O [65]. This section analyzes the time complexity of SCCOA. The time complexity of the added exploration and exploitation mechanisms is $O(N \times D \times T)$. The time complexity of the mutation mechanism is $O(N \times C \times T)$. Therefore, the overall time complexity of SCCOA is $O(N \times (C + D) \times T)$. The time complexity of the original COA algorithm is also $O(N \times (C + D) \times T)$. SCCOA adds three strategies to the original algorithm: sinusoidal and cosine fluctuation exploration and exploitation, balanced exploration

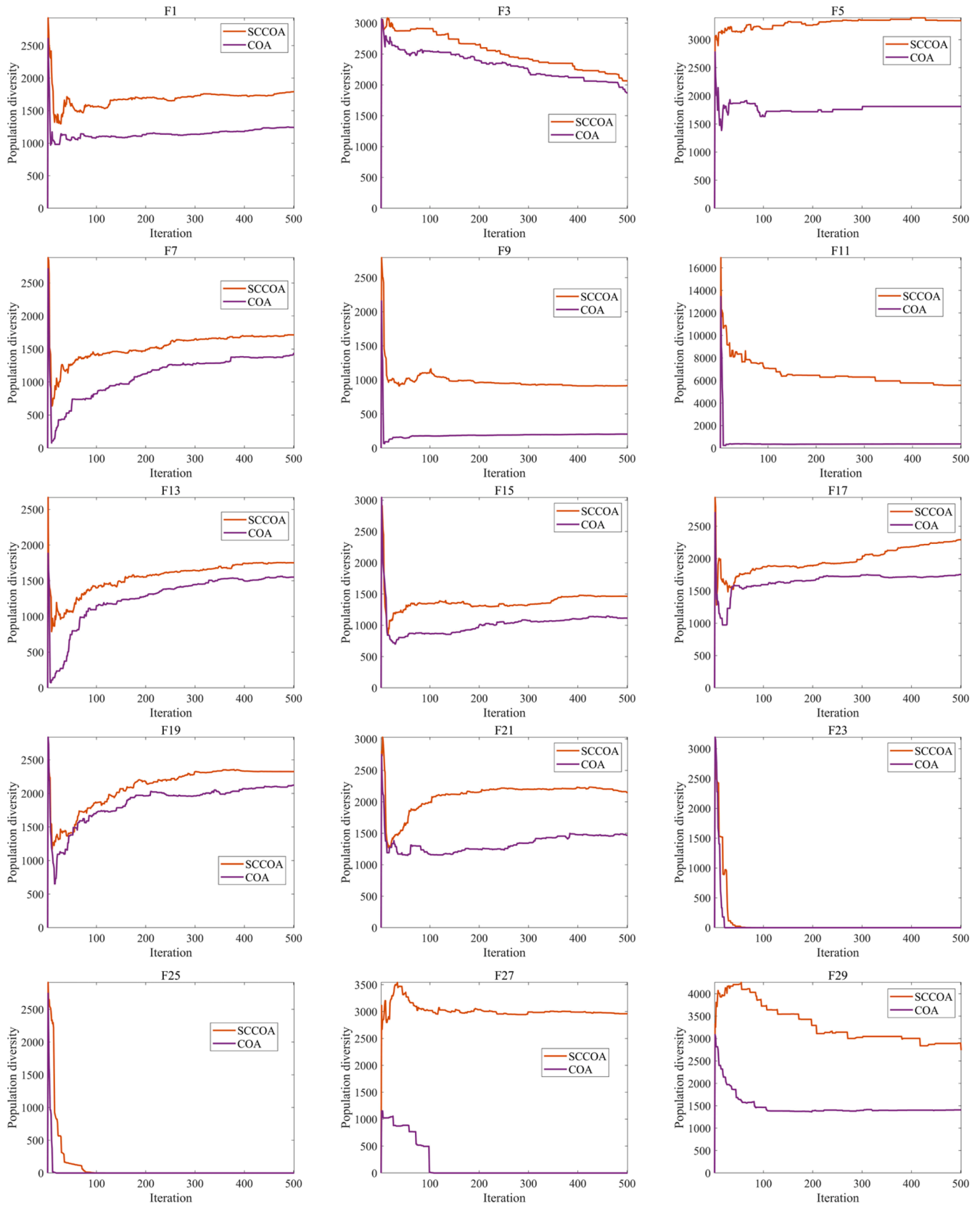


Fig. 3. COA and SCCOA population diversity analysis.

and exploitation weight, and mutation perturbation operator. The time complexity remains unchanged compared to the original algorithm, so the improved SCCOA runs at the same speed as the original COA.

4. Novel multi-robot leader-follower formation control model

In the leader-follower formation control problem, one robot is selected as the leader, which is mainly responsible for the path planning task of the entire formation. In contrast, the remaining robots are regarded as followers. The follower robots are responsible for tracking the leader and maintaining the required distance and angle with the leader as much as possible, so as to ensure that the entire multi-robot formation moves in accordance with the expected ideal formation without collision and finally reaches the target position [66].

The adopted hierarchical leader-follower architecture is shown in Fig. 4. The framework mainly consists of the following 5 parts.

- (1) The navigator dynamics are shown in Eq. (37).

$$\dot{P}_L = v_L^{opt} + K_g(g - P_L) + F_{obs} \quad (37)$$

where, v_L^{opt} is the navigator velocity generated by the SCCOA algorithm, K_g is the navigation gain, g is the target position, P_L is the navigator position, and F_{obs} is the velocity disturbance generated by obstacle avoidance.

- (2) The follower control law is shown in Eq. (38).

$$v_i = v_L \otimes A_i + K_f(\Delta_i - (P_i - P_L)) + F_{obs} \quad (38)$$

where, $\otimes$ represents the domain velocity fusion, K_f is the error tracking gain, and Δ_i is the expected deviation.

- (3) The environmental space is shown in Eq. (39).

$$\mathcal{O} = \bigcup_{j=1}^M \{p \in \mathbb{R}^2 \mid \|p - o_j\| \leq r_j\} \quad (39)$$

where, o_j is the obstacle center, r_j is the obstacle radius, the expansion obstacle model is used for safety distance constraint, and the potential field function is designed as Eq. (40).

$$\phi(p, o_j) = \begin{cases} k_{obs} \left(\frac{1}{d} - \frac{1}{d_{safe}} \right)^2 & d < d_{safe} \\ 0 & otherwise \end{cases} \quad (40)$$

where, $d = \|p - o_j\| - r_j$ represents the difference between the distance between the agent and the obstacle and the obstacle radius. d_{safe} is the safety distance. k_{obs} is the dynamic exclusion coefficient, which will be adjusted according to the distance and density of obstacles during the obstacle avoidance process. It increases when approaching the obstacle and decreases when moving away from it, thereby improving the real-time performance and accuracy of obstacle avoidance, as shown in Eq. (41):

$$k_{obs} = \alpha_q \left(\frac{1}{d} - \frac{1}{d_{safe}} \right) + \alpha_v u_{ro} + \alpha_a a_{ro} \quad (41)$$

where, α_q , α_v , and α_a represent the weight coefficients of each component, which are used to adjust the contribution of different factors to the repulsive force. u_{ro} represents the difference between the velocity vector of the agent and the obstacle. a_{ro} represents the difference between the acceleration vector of the agent and the obstacle.

- (4) Communication topology

The dynamic adjacency matrix satisfies Eq. (42).

$$A_{ij}(t) = \begin{cases} 1 & \|p_i - p_j\| \leq R_c \\ 0 & otherwise \end{cases} \quad (42)$$

where, R_c is the communication radius and $\|p_i - p_j\|$ is the distance between agents.

The communication quality factor is introduced to optimize the information weight, and its mathematical description is shown in Eq. (43).

$$Q_{ij} = e^{(-\lambda \|p_i - p_j\|)} \quad (43)$$

where, λ is the attenuation coefficient. The longer the distance, the lower the communication weight.

Through the study of fuzzy control technology [67], the probabilistic connection disturbance factor is added to the communication topology

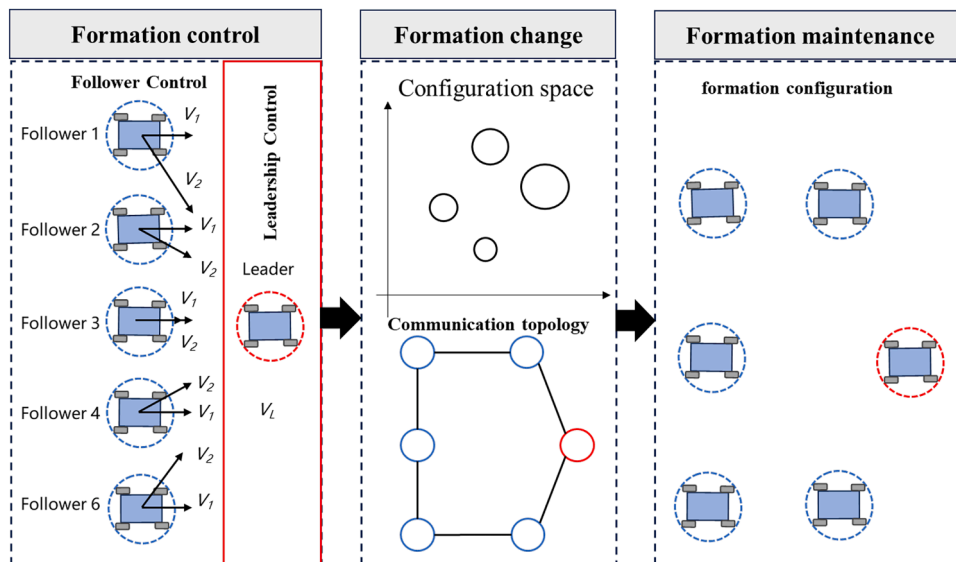


Fig. 4. Hierarchical leader-follower formation framework.

model to enhance the applicability of the communication topology model. Its mathematical model is shown in Eq. (44).

$$w_{ij} = f\left(\frac{P_{ij}C_{ij}G_{ij}}{\sigma^2}\right) \times f\left(\theta_1, \|p_j - p_i\|\right) \quad (44)$$

where, P_{ij} is the formation efficiency, C_{ij} is the communication gain, G_{ij} is the channel gain, σ^2 is the signal disturbance power, and θ_1 is the relative angle between the formation control individuals. In order to further verify the superiority of the algorithm in path planning, σ^2 is set to the maximum interference value within the communication range.

(5) Formation configuration

The polygonal formation is constructed using rigid graph theory. The expected relative position set $\{\Delta_i\}$ satisfies Eq. (45).

$$\sum_{i=1}^{N_F} \Delta_i = 0 \quad (45)$$

where, N_F is the number of followers.

4.1. Hierarchical consistency control strategy

4.1.1. Leader control layer

The primary function of the navigator's control is global path planning. The proposed SCCOA algorithm balances navigation and obstacle avoidance to achieve optimal path planning. Furthermore, adaptive weight parameters, weighted reference speed, and target attraction are used to achieve global path optimization and motion control for the navigator.

This paper considers multiple velocity sources to ensure the integrity of the navigator's motion. Its velocity synthesis term includes the optimization term, the reference velocity term, the target attraction term, and the obstacle repulsion term. The navigator's control design and its dynamic equation are shown in Eq. (46).

$$v_L = \underbrace{\rho_1 \frac{p_{opt} - p_L}{\Delta t}}_{\text{optimizationterm}} + \underbrace{\rho_2 \cdot v_{ref}}_{\text{referencevelocityterm}} + \underbrace{\rho_3 \cdot v_{goal}}_{\text{targetattractionterm}} + \underbrace{w_{obs} \cdot \nabla \phi_{obs}}_{\text{obstacle repulsionterm}} \quad (46)$$

(1) Optimization term

The optimization term represents the weighted velocity after SCCOA optimization. This term ensures the agent accurately searches for relatively low-cost paths during multi-step execution. It also optimizes the agent's obstacle avoidance performance to a certain extent through the use of a penalty function. p_{opt} represents the position of the leader after SCCOA optimization, p_L represents the position of the leader before optimization, and ρ_1 represents the weight parameter.

The cost function of the SCCOA algorithm applied to the leader is Eq. (47).

$$J_L = w_1 \|p_L - g\|^2 + w_2 \sum_{j=1}^M \phi(p_L, o_j) \quad (47)$$

where, $\phi(p_L, o_j)$ is the obstacle avoidance potential field function, w_1 and w_2 are weight coefficients used to balance the priority of navigation and obstacle avoidance.

$$\phi(p_L, o_j) = \tanh\left(\frac{1}{d_j + \epsilon}\right) \quad (48)$$

where, d_j represents the distance between the navigator and the obstacle.

(2) Reference velocity

v_{ref} is the preset velocity, which ensures that the agent's velocity is within the appropriate velocity range at each time step, preventing its speed from spanning too large a range. ρ_2 is the weight parameter.

(3) Target attraction term

v_{goal} is the pulling speed of the target point, ensuring that the agent does not deviate from the target. This speed decreases proportionally as the distance between the agent and the target point decreases. ρ_3 is a fixed weight parameter.

$$v_{goal} = v_{max} \cdot \frac{p_{goal} - p_L}{\|p_{goal} - p_L\|} \quad (49)$$

(4) Obstacle repulsion term

The obstacle repulsion term compensates for the optimization term, preventing the agent from degrading its obstacle avoidance performance to ensure path economy. This term strengthens the obstacle avoidance function of the entire system. The directional field gradient is $\nabla \phi_{obs}$, and the weight is w_{obs} . $\nabla \phi$ is the repulsive potential field gradient function, mathematically described as Eq. (50).

$$\nabla \phi_j = \begin{cases} \frac{p - o_j}{\|p - o_j\|^3} \cdot \left(\frac{1}{\|p - o_j\| - r_j} - \frac{1}{3.0} \right), & \|p - o_j\| < 3.0 + r_j \\ 0, & \text{else} \end{cases} \quad (50)$$

where, p is the agent's position, o_j is the obstacle's position, and r_j is the obstacle's radius. As the agent moves away from the obstacle and closer to the target point, w_{obs} gradually decays to 0.

4.1.2. Follower control layer (consistency-optimization hybrid control)

The follower control law design and the follower's dynamic equation are:

$$v_i(k+1) = \underbrace{\gamma_1 v_{desired}}_{\text{optimizationterm}} + \underbrace{\gamma_2 v_{formation}}_{\text{consensusterm}} + \underbrace{f_{obs_comp,i}(k)}_{\text{obstaclecompensationterm}} \quad (51)$$

where, γ_1 and γ_2 are weight parameters, and the remaining components are as follows.

(1) Optimization term (SCCOA algorithm)

$$v_{desired} = \frac{p_{opt,i}(k) - p_i(k)}{\Delta t} \quad (52)$$

where, $p_{opt,i}$ is obtained after optimization by the SCCOA algorithm, p_i is the speed before optimization, and the mixed cost function of the follower applying the SCCOA algorithm is:

$$J_i = w_{form} \|p - (p_L + \delta_i)\| + w_{obs} \sum \exp\left(-\frac{\|p - o_j\|}{d_{safe}}\right) \quad (53)$$

where, w_{form} and w_{obs} are weight parameters, and d_{safe} is the safe distance between the agent and the obstacle.

(2) Consensus term

The consensus term of the follower velocity is:

$$v_{formation} = \eta_1 v_L(k) + \eta_2 \frac{(p_L(k) + \delta_i) - p_i(k)}{\Delta t} \quad (54)$$

where, η_1 and η_2 are fixed weights, with a value of 0.5.

(3) Obstacle compensation term

The obstacle compensation term is:

$$f_{obs_comp,i} = \gamma_3 \cdot \hat{f}_{obs,i} \quad (55)$$

where, $\hat{f}_{obs,i}$ is a unit direction vector, its modulus $\|\hat{f}_{obs,i}\|$ is 1, γ_3 is a dynamic weight coefficient, its value does not exceed 1. $f_{obs,i}$ is the potential field function, defined as:

$$f_{obs,i} = \sum_{j=1}^M \frac{p_i - o_j}{\|p_i - o_j\|} \cdot \exp\left(-\frac{\|p_i - o_j\| - r_j}{1.5}\right) \quad (56)$$

where, M is the number of obstacles.

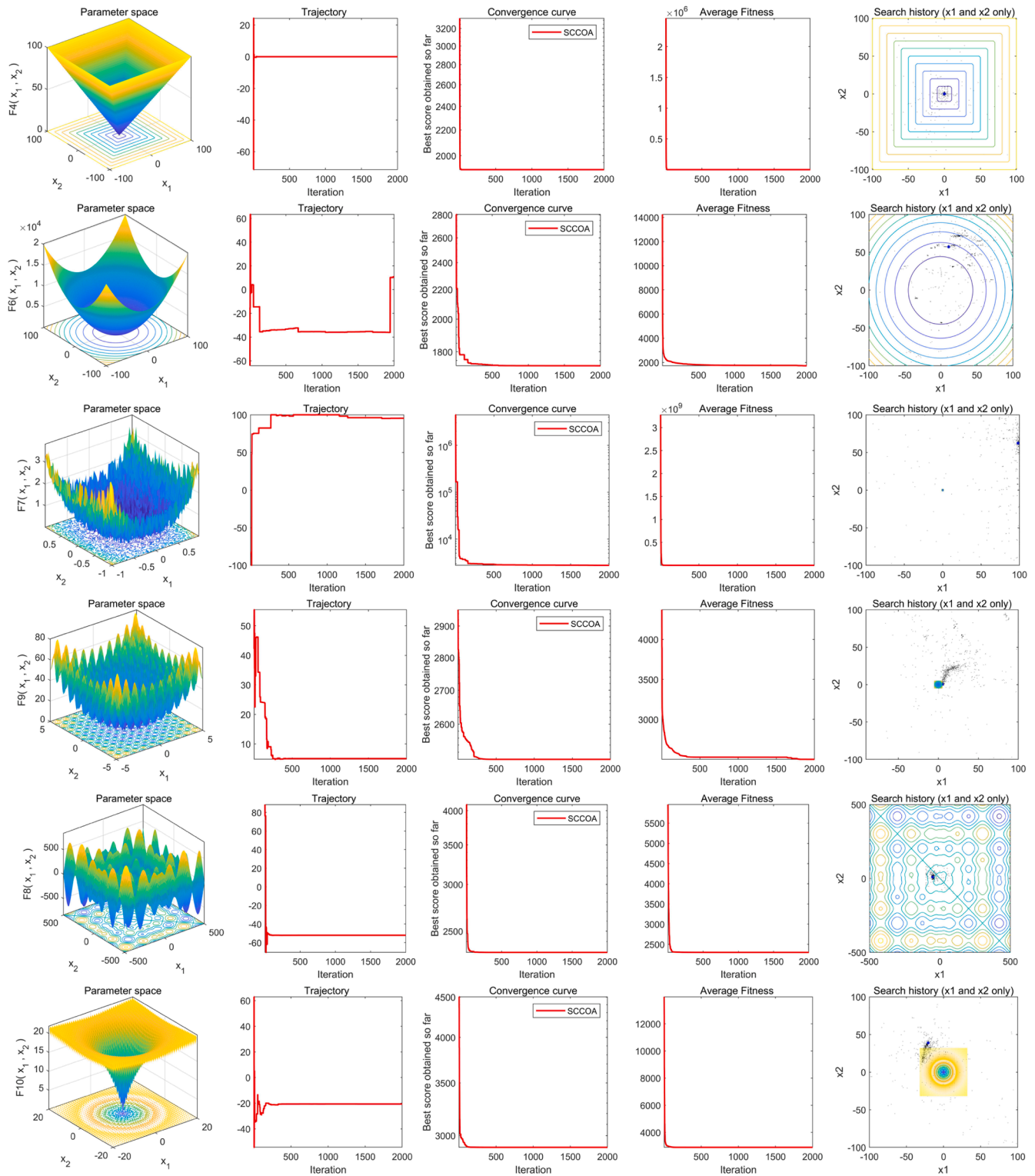


Fig. 5. SCCOA convergence behavior analysis diagram.

4.2. Proof of stability

(1) Formation error dynamics

Substituting the formation error definition (Eq. (57)) into the dynamic model, we can obtain the error equation:

$$\begin{aligned} e_i(k+1) &= p_L(k+1) + \delta_i - p_i(k+1) \\ &= [p_L(k) + v_L(k)\Delta t] + \delta_i - [p_i(k) + v_i(k+1)\Delta t] \\ &= e_i(k) + [v_L(k) - v_i(k+1)]\Delta t \end{aligned} \quad (57)$$

$$\begin{aligned} e_i(k+1) &= e_i(k) + \left[v_L(k) - 0.6 \frac{p_{opt,i}(k) - p_i(k)}{\Delta t} - 0.4 \left(0.5v_L(k) + 0.5 \frac{e_i(k)}{\Delta t} \right) - \gamma_3 f_{obs,i}(k) \right] \Delta t \\ &= e_i(k)(1 - 0.4 \times 0.5) + \left[v_L(k) - 0.6 \frac{p_{opt,i}(k) - p_i(k)}{\Delta t} - 0.4 \times 0.5v_L(k) \right] \Delta t - \gamma_3 f_{obs,i}(k)\Delta t \\ &= 0.8e_i(k) + 0.2v_L(k)\Delta t - 0.6 \left(p_{opt,i}(k) - p_i(k) \right) - \gamma_3 f_{obs,i}(k)\Delta t \end{aligned} \quad (62)$$

Substitute the control law into the error equation and take appropriate parameter values, γ_1 is 0.6, γ_2 is 0.4. Define the error dynamics:

$$\begin{aligned} e_i(k+1) &= e_i(k) + \left[v_L(k) - \left(0.6 \frac{p_{opt,i}(k) - p_i(k)}{\Delta t} + 0.4 \left(0.5v_L(k) \right. \right. \right. \\ &\quad \left. \left. \left. + 0.5 \frac{e_i(k)}{\Delta t} \right) + \gamma_3 f_{obs,i}(k) \right) \right] \Delta t \end{aligned} \quad (58)$$

After simplification:

$$\begin{aligned} e_i(k+1) &= \underbrace{0.8e_i(k) + 0.2 \left[p_L(k) + \delta_i - p_{opt,i}(k) \right]}_{\text{formation tracking term}} - \underbrace{0.6 \left[p_{opt,i}(k) - p_i(k) \right]}_{\text{optimization adjustment}} \\ &\quad - \underbrace{\gamma_3 f_{obs,i}(k)\Delta t}_{\text{disturbance term}} \end{aligned} \quad (59)$$

(2) Assumption conditions

H1: The optimization algorithm satisfies:

$$\lim_{k \rightarrow \infty} \| p_{opt,i}(k) - (p_L(k) + \delta_i) \| = 0$$

H2: The obstacle repulsion effect is bounded, that is, it satisfies:

$$\| f_{obs_comp,i} \| = \gamma_3 \| f_{obs,i} \| \leq \gamma_3 \| e_i(k) \|$$

Regarding H1, due to the random search structure of metaheuristic algorithms, their convergence cannot be strictly proved mathematically. However, their greedy mechanism allows metaheuristic algorithms to converge to arbitrary precision when computing resources are sufficient and parameters are adjustable.

(3) Effective convergence verification

To verify the algorithm's probabilistic stability in path planning, the algorithm's effective convergence rate was analyzed. Fig. 5 shows the convergence behavior analysis of the proposed algorithm, SCCOA. The convergence curve in the convergence behavior analysis shows that SCCOA converges quickly in the early stages of path planning and obtains the optimal solution. There is no subsequent fluctuation interference in the intelligent agent control of the path planning, and real-time stability control is performed on the moving individuals after path planning.

(4) Lyapunov function construction

Define the candidate function:

$$V(k) = \sum_{i=1}^{N-1} e_i(k)^T e_i(k) \quad (60)$$

Difference calculation:

$$\Delta V(k) = V(k+1) - V(k) = \sum_{i=1}^{N-1} [e_i(k+1)^T e_i(k+1) - e_i(k)^T e_i(k)] \quad (61)$$

Substitute the error equation and expand it:

(5) Derivation of stability conditions

It is known that H1 holds. When k is large enough, the SCCOA algorithm makes $p_{opt,i}(k) \rightarrow p_L(k) + \delta_i$, that is:

$$p_{opt,i}(k) - p_i(k) \approx e_i(k) \quad (63)$$

After substitution, the error equation can be simplified to:

$$\begin{aligned} e_i(k+1) &= 0.8e_i(k) + 0.2v_L(k)\Delta t - 0.6e_i(k) - \gamma_3 f_{obs,i}(k)\Delta t \\ &= 0.2e_i(k) + 0.2v_L(k)\Delta t - \gamma_3 f_{obs,i}(k)\Delta t \end{aligned} \quad (64)$$

(6) Differential analysis

$$\| f_{obs_comp,i}(k) \| \leq \gamma_3 \| e_i(k) \| \quad (65)$$

then the Lyapunov difference satisfies:

$$\Delta V(k) \leq \sum_{i=1}^{N-1} [(0.2^2 - 1) \| e_i(k) \|^2 + 0.2\gamma_3\Delta t \| e_i(k) \| + \gamma_3^2\Delta t^2 \| e_i(k) \|^2] \quad (66)$$

given $\gamma_3 \leq 1$, it is obvious that there exists α such that:

$$\Delta V(k) \leq -\alpha V(k) \quad (67)$$

According to the above proof, the formation system is asymptotically stable.

4.3. Performance evaluation metrics

This paper uses path length, formation keeping error, and formation speed tracking error as quantitative indicators to verify the algorithm's performance.

(1) Path length

The cumulative sum of the Euclidean distances of each agent's trajectory from the starting point to the end point is calculated as follows:

$$L_i = \sum_{k=1}^T \sqrt{(x_i^{k+1} - x_i^k)^2 + (y_i^{k+1} - y_i^k)^2} \quad (68)$$

where, T is the effective simulation deployment, which reflects the economy of path planning.

(2) Formation maintenance error

The deviation modulus between the follower's actual position and the expected formation position is defined as:

$$e_{form}^i(t) = \| \mathbf{p}_i(t) - (\mathbf{p}_L(t) + \delta_i) \| \quad (69)$$

where, δ_i is the relative coordinate of the formation, and its root mean square error is calculated to evaluate the stability of the formation.

(3) Formation speed tracking error

The velocity difference between the follower and the leader, including the x and y direction components and the resultant velocity modulus error:

$$e_{v_x}^i(t) = v_{x,i}(t) - v_{x,L}(t), \quad e_{v_y}^i(t) = v_{y,i}(t) - v_{y,L}(t) \\ e_v^i(t) = \sqrt{(e_{v_x}^i(t))^2 + (e_{v_y}^i(t))^2} \quad (70)$$

The root mean square value was calculated to measure the consistency of the collaborative motion.

5. Path planning and formation control experiments

5.1. Experimental setup

The experiment was conducted using the MATLAB 2021a simulation platform. A multi-agent system consisting of one leader and five followers (a total of 6 agents) was constructed. The simulation lasted 40 seconds, with a control period of 0.1 seconds. The agents were initially distributed, with the leader at the origin and the followers at initial coordinates of (5.0, 0.0), (-2.0, 3.0), (-2.0, -3.0), (4.0, -4.0), and (4.0, 4.0). The target coordinates were (40, 35).

The obstacles are set as 7 circular static obstacles, and their information matrix is:

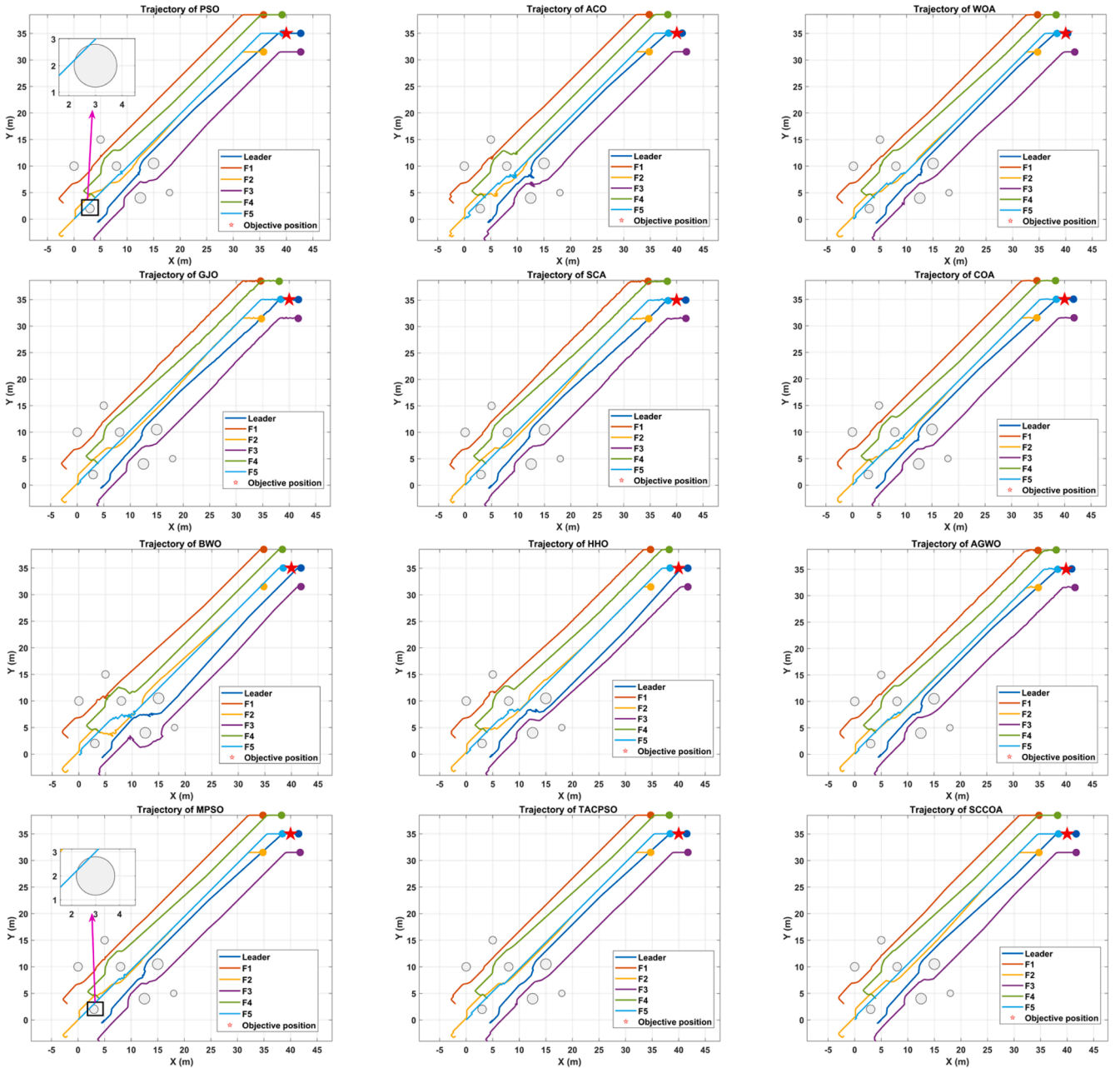


Fig. 6. Formation trajectories after optimization by various algorithms.

$$\text{obstacles} = \begin{bmatrix} 3.0 & 2.0 & 0.8 \\ 0 & 10 & 0.8 \\ 12.5 & 4.0 & 1.0 \\ 15.0 & 10.5 & 1.0 \\ 5.0 & 15.0 & 0.7 \\ 8.0 & 10.0 & 0.8 \\ 18.0 & 5.0 & 0.61 \end{bmatrix} \quad (71)$$

The first and second columns represent the obstacle's center position, and the third column represents its radius.

The adjacency matrix at the initial moment is:

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 \end{bmatrix} \quad (72)$$

Other parameters include: communication range R_c is 8 meters(m), maximum moving speed is 1.8m/s, population size of SCCOA algorithm is 50, and number of iterations is 30.

5.2. Formation trajectory analysis

A comparative approach was employed to validate the superior performance of SCCOA in multi-robot path planning and formation control applications. SCCOA was compared with the original COA algorithm and various leading metaheuristic algorithms in path planning and formation control experiments, demonstrating its superior performance. PSO, HHO, and SCA are classic comparison algorithms commonly used in recent years, demonstrating superior optimization results in various areas. ACO, WOA, GJO, and BWO are excellent biological population optimization algorithms often used for algorithm comparison and particularly prominent in certain fields. AGWO, MPSTO, and TACPSO are improved optimization algorithms based on these excellent biological optimization algorithms, demonstrating even better performance. Comparisons with these selected comparison algorithms demonstrate the superior performance of SCCOA from multiple dimensions and perspectives. By comparing with the above-selected comparison algorithms, the excellent performance of SCCOA is reflected from multiple dimensions and aspects. Each algorithm is run 20 times under the same operating environment. Finally, the obtained data was statistically analyzed and verified using the Wilcoxon rank-sum test (WT) [68]. In the Wilcoxon rank-sum test, the '+' symbol indicates that the algorithm mentioned is superior to the comparison algorithm, the '-' symbol indicates that the algorithm mentioned is inferior to the comparison algorithm, and the '=' symbol indicates that there is no significant difference between the mentioned algorithm and the comparison algorithm.

The planned trajectory of the formation system reflects the formation's immediate response to obstacles. The system must ensure safety during both formation changes and formation maintenance. This means considering the location of obstacles and endpoints during formation control, ensuring the formation's overall path is safe and economical. Fig. 6 shows the optimized formation trajectories of 12 algorithms.

Fig. 6 shows that the trajectory of F5 (the fifth follower in the

formation) obtained by the PSO and MPSTO algorithms crosses an obstacle. This agent does not perform any obstacle avoidance actions before encountering the obstacle, indicating that these algorithms' search performance is insufficient, resulting in an inability to find the optimal solution at the obstacle's location. Furthermore, while maintaining obstacle avoidance performance, SCCOA achieves a smoother path than the other algorithms, demonstrating a better balance between exploration and exploitation and superior performance.

Table 2 shows the total optimized path lengths for each algorithm. While the TACPSO algorithm achieves the shortest path length (322.93 m), it suffers from insufficient search performance and, given limited population and computational resources, is more susceptible to local optima than other algorithms. The SCCOA algorithm achieves a path length of 324.70 m. While this is slightly inferior to the TACPSO and MPSTO (323.29 m) in terms of path length, its obstacle avoidance performance and path smoothness are superior to those of the other algorithms.

5.3. Formation speed error analysis

Table 3 shows the root means square values of the x-direction velocity errors for the 12 algorithms. The data in this table indicates that SCCOA has a root means square x-direction velocity error of 0.5972, the lowest value among the 12 algorithms, and achieves a '+' rating in the WT test compared to all other compared algorithms. Throughout the formation control experiment, SCCOA exhibited minimal x-direction velocity fluctuations and converged faster than the other algorithms.

Fig. 7 shows the x-direction velocity error curves for each optimized algorithm. It can be seen that SCCOA exhibits smaller overall velocity fluctuations than the other algorithms. WOA, GJO, SCA, HHO, and AGWO still exhibit significant velocity fluctuations in the later stages of the formation system operation, indicating that these algorithms have not yet achieved velocity consistency and lack optimality and stability. Among the algorithms that achieve velocity consistency within a finite timeframe, the SCCOA-controlled formation is the first to achieve x-direction velocity consistency, demonstrating the stability and optimization efficiency of the SCCOA algorithm.

Table 4 shows the root mean square values of velocity errors in the y-direction for the 12 algorithms. The data in this table shows that SCCOA achieves a y-direction root mean square value of 0.378, the lowest among the 12 algorithms. Unlike the root mean square values for velocity errors in the x-direction, SCCOA performs even more effectively. Compared to BWO (0.8315), the algorithm with the largest root means square velocity error, SCCOA achieves a nearly 50 % reduction in y-direction velocity error, significantly reducing it compared to the other algorithms. This demonstrates that SCCOA converges faster than the other algorithms.

Fig. 8 shows the velocity error curves for the optimized formations in the y-direction. It can be seen that SCCOA exhibits less overall velocity fluctuation than the other algorithms. WOA, GJO, SCA, COA, HHO, and AGWO still exhibit significant velocity fluctuations in the later stages of the formation system operation, indicating that these algorithms have not yet achieved velocity consistency and lack optimality and stability. Among the algorithms that achieve velocity consistency within a finite timeframe, although the SCCOA-controlled formation does not achieve

Table 2
Total path length after optimization of each algorithm.

		PSO	ACO	WOA	GJO	SCA	COA
Total path length		330.79	341.67	326.34	327.23	327.96	326.01
	P	6.7956E-08	6.7956E-08	1.2009E-06	6.7956E-08	6.7956E-08	6.7956E-08
	Wr	+	+	+	+	+	+
	BWO		HHO	AGWO	MPSTO	TACPSO	SCCOA
		345.33	326.77	325.90	323.29	322.93	324.70
	P	6.7956E-08	5.1658E-06	4.6804E-05	2.0616E-06	4.6804E-05	—
	Wr	+	+	+	-	-	—

Table 3
Root means square error of x-direction velocity after optimization of each algorithm.

		PSO	ACO	WOA	GJO	SCA	COA
x-direction velocity root means square error		0.6917	0.8401	0.6583	0.6536	0.6681	0.6982
P	Wr	6.7956E-08	6.7956E-08	6.7956E-08	6.7956E-08	6.7956E-08	6.7956E-08
		+	+	+	+	+	+
P	BWO	0.8499	0.6807	0.6825	0.6534	0.6515	0.5972
	Wr	6.7956E-08	6.7956E-08	6.7956E-08	6.7956E-08	6.7956E-08	—
P	Wr	+	+	+	+	+	—

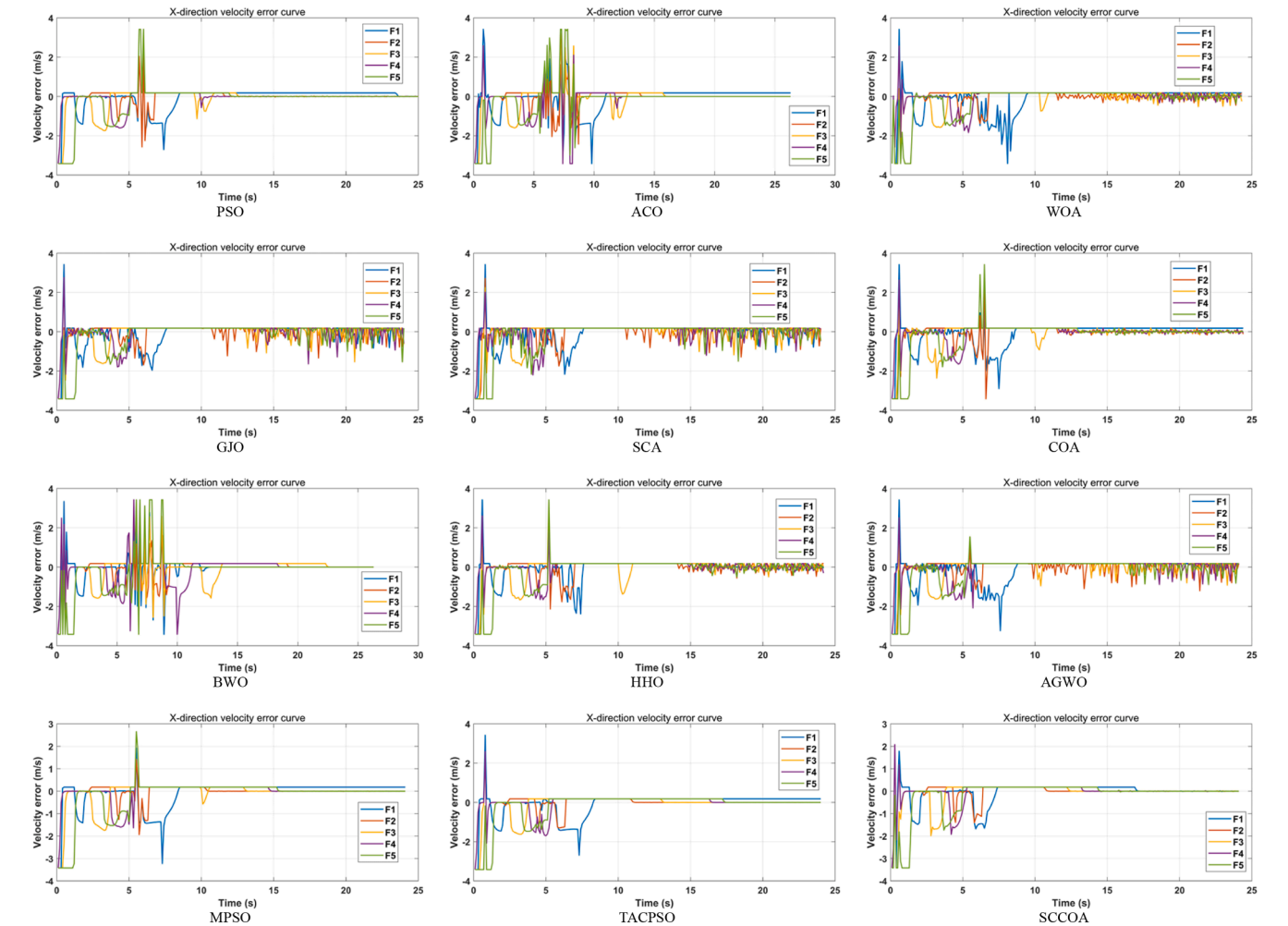


Fig. 7. Velocity error curves of the formation in the x-direction after optimization by each algorithm.

Table 4
Root means square error of y-direction velocity after optimization of each algorithm.

		PSO	ACO	WOA	GJO	SCA	COA
y-direction velocity root means square error		0.4996	0.7523	0.6358	0.5881	0.6260	0.5607
P	Wr	6.7956E-08	6.7956E-08	6.7956E-08	6.7956E-08	6.7956E-08	6.7956E-08
		+	+	+	+	+	+
P	BWO	0.8315	0.6516	0.6631	0.5339	0.4946	0.3780
	Wr	6.7956E-08	6.7956E-08	6.7956E-08	6.7956E-08	6.7956E-08	—
P	Wr	+	+	+	+	+	—

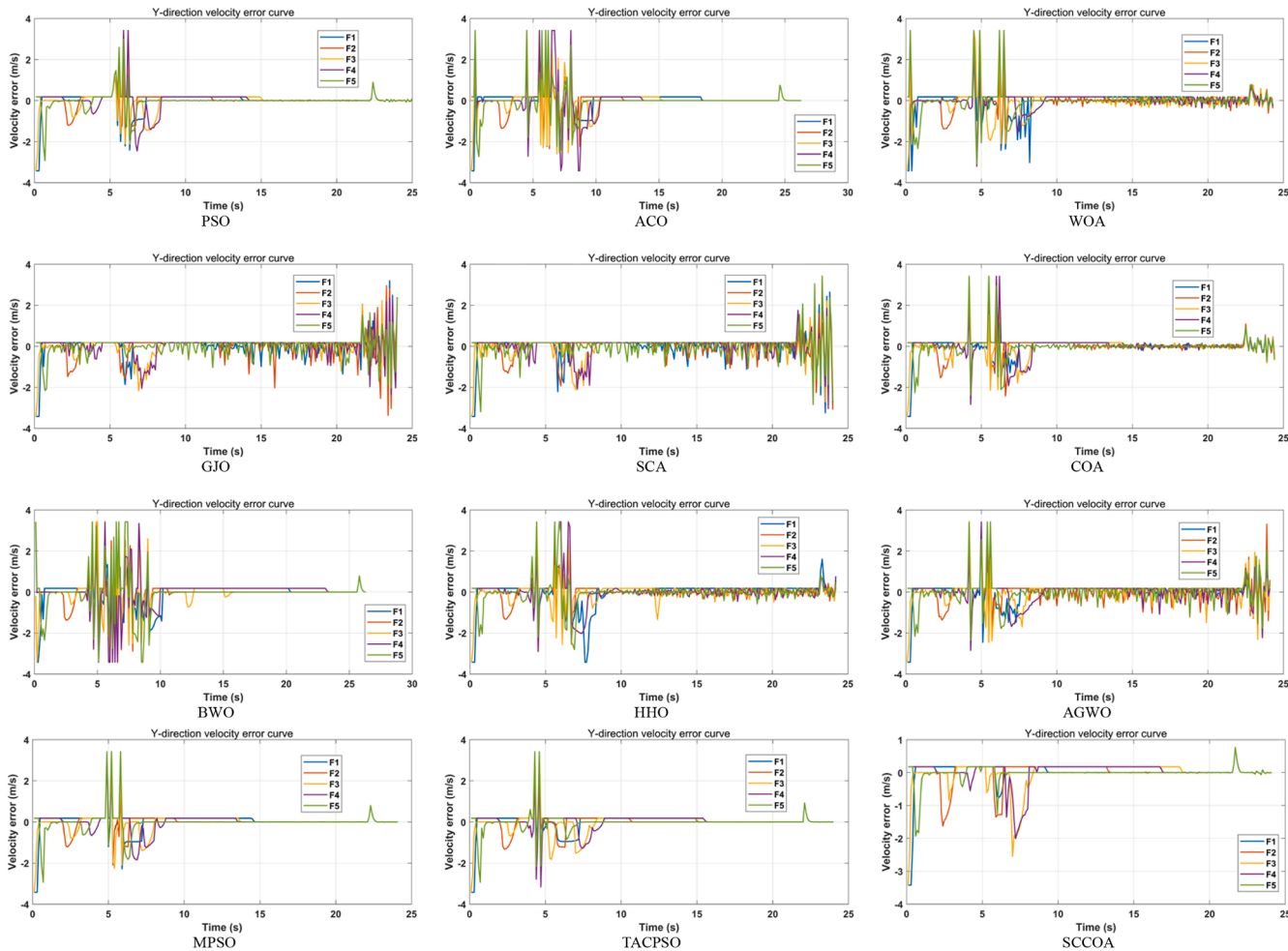


Fig. 8. Velocity error curves of the formation in the y direction after optimization by each algorithm.

Table 5
Root means square value of formation position error after optimization of each algorithm.

		PSO	ACO	WOA	GJO	SCA	COA
Formation position root means square error	P	0.9717	1.1750	1.4728	1.0101	1.0009	1.0493
	Wr	1.0141E-03	1.2009E-06	6.7956E-08	6.7956E-08	6.7956E-08	6.7956E-08
		BWO	HHO	AGWO	MPSO	TACPSO	SCCOA
	P	1.1137	1.0844	1.3001	1.1443	1.2089	0.9324
	Wr	6.7956E-08	6.7956E-08	6.7956E-08	6.7956E-08	6.7956E-08	—
		+	+	+	+	+	—

Table 6
Root means square value of formation speed synthesis error of each algorithm.

		PSO	ACO	WOA	GJO	SCA	COA
Velocity composite error root mean square	P	0.8533	1.1277	0.9152	0.8792	0.9155	0.8955
	Wr	6.7956E-08	6.7956E-08	6.7956E-08	6.7956E-08	6.7956E-08	6.7956E-08
		BWO	HHO	AGWO	MPSO	TACPSO	SCCOA
	P	1.1890	0.9424	0.9516	0.8438	0.8179	0.7068
	Wr	6.7956E-08	6.7956E-08	6.7956E-08	6.7956E-08	6.7956E-08	—
		+	+	+	+	+	—

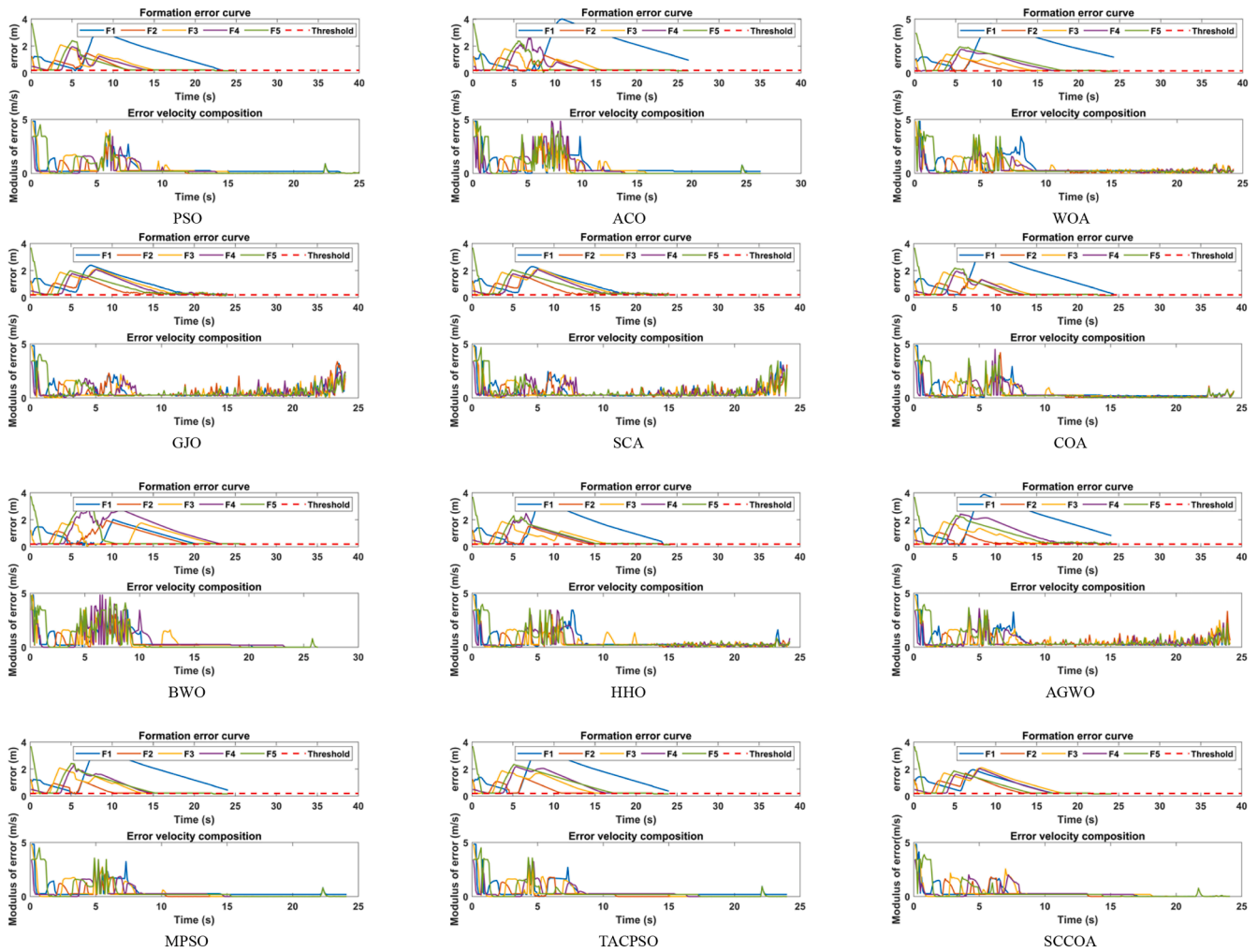


Fig. 9. Formation error change curve and speed composite error curve after optimization of each algorithm.

velocity consistency in the y-direction first, its root mean square error throughout the entire process is smaller than that of TACPSO, demonstrating the SCCOA algorithm's global stability and optimization efficiency.

5.4. Formation position error analysis

In the formation position error analysis, SCCOA has the smallest root mean square value (0.9324) and achieves a '+' in the WT test, demonstrating its superiority. Table 5 shows the root mean square values of the formation position errors for each algorithm.

Table 6 shows the root mean square values of the formation velocity synthesis errors of the 12 algorithms. From the velocity error data in the x and y directions, it can be seen that SCCOA has the smallest velocity synthesis error among the 12 compared algorithms.

Fig. 9 shows the formation position error and velocity composite error curves for the 12 algorithms, which more clearly demonstrates the comparison between velocity error and formation position error. It is clear that SCCOA's formation position error and velocity composite error converge first.

6. Conclusions and future works

Existing optimization algorithms for multi-robot system path planning and formation control suffer from shortcomings such as excessive

complexity and insufficient optimization performance, leading to low efficiency and high computational resource consumption in formation control. This paper proposes a hybrid algorithm SCCOA for multi-robot system path planning and formation control based on COA. By analyzing the path planning and formation control techniques of multi-robot systems, a hierarchical leader-follower formation framework was designed. Comparative experiments were then conducted, and based on the analysis of various coefficients in multi-robot path planning and formation control, a variety of performance evaluation metrics were designed to assess the effectiveness of the algorithm. Experimental results show that SCCOA ranks first in all four evaluation metrics and achieves the lowest trajectory, velocity, and position errors compared to other leading algorithms. Finally, error data and visual graph analysis demonstrate the effectiveness and advancement of SCCOA in solving formation control problems.

Although the proposed algorithm has achieved considerable results in the experiments presented in this paper, it still has some shortcomings in path trajectory planning and cannot balance optimization speed and results in some processes, causing slight errors in the multi-robot system path planning and formation control outcomes. Therefore, future work will focus on further improving the algorithm and multi-robot system path planning and formation control techniques, applying them to more complex multi-robot system frameworks and formation problems; additionally, efforts will be made to establish an experimental platform and carry out physical tests to further verify the algorithm's

performance in real control environments, promoting its application in practical engineering problems, such as multi-target inspection robots in industrial and security fields, multi-target vehicle ride-sharing scheduling optimization, and multi-target drone delivery route optimization.

Data availability

The data that support the findings of this study are available from the corresponding author upon reasonable request.

CRediT authorship contribution statement

Liguo Yao: Writing – original draft, Visualization, Software, Formal analysis, Conceptualization. **Xiaoyang Yuan:** Writing – review & editing, Methodology, Conceptualization. **Guanghui Li:** Writing – review & editing, Formal analysis, Visualization. **Yao Lu:** Supervision, Methodology. **Taihua Zhang:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the Guizhou Provincial Basic Research Program (Natural Science) (Grant No. Qiankehejichu-ZK[2024] General 424), National Natural Science Foundation of China (Grant No.72061006), Guizhou Provincial Basic Research Program (Natural Science) General Project (Grant No. Qiankehejichu-MS[2025]246) and Science and Technology Platform Project of Guizhou Province, China (Grant No. ZSYS[2025]011).

Data availability

Data will be made available on request.

References

- [1] R. Bao, J. Tao, J. Zhao, M. Dong, J. Li, C. Pan, Integrated intelligent tactile system for a humanoid robot, *Sci. Bull.* 68 (10) (2023) 1027–1037, <https://doi.org/10.1016/j.scib.2023.04.019>.
- [2] I. Jakob, A. Kollreider, M. Germanotta, F. Benetti, A. Cruciani, L. Padua, I. Aprile, Robotic and sensor technology for upper limb rehabilitation, *PM&R* 10 (9) (2018) S189–S197, <https://doi.org/10.1016/j.pmrj.2018.07.011>. Supplement 2.
- [3] Y. Luo, M. Zhang, Y. Liu, J. Lin, Z. Zhang, Dynamic neural learning for obstacle avoidance of humanoid robot performing cooperative tasks, *Neurocomputing* 633 (2025) 129727, <https://doi.org/10.1016/j.neucom.2025.129727>.
- [4] L. Yao, G. Li, T. Zhang, A.G. Hussien, Y. Lu, Adaptive multi-step path planning for multi-robot in dynamic environments based on hybrid optimization approach, *Expert. Syst. Appl.* 298 (2026) 129699, <https://doi.org/10.1016/j.eswa.2025.129699>.
- [5] S. Wang, L. Jiang, J. Meng, Y. Xie, H. Ding, Training for smart manufacturing using a mobile robot-based production line, *Front. Mech. Eng.* 16 (2) (2021) 249–270, <https://doi.org/10.1007/s11465-020-0625-z>.
- [6] N. Zaerpour, R. Volbeda, A. Gharehgozli, Automated or manual storage systems: do throughput and storage capacity matter? *INFOR: Inf. Syst. Oper. Res.* 57 (1) (2019) 99–120, <https://doi.org/10.1080/03155986.2018.1532765>.
- [7] S.J. Jung, R. Myllylä, W.Y. Chung, Wireless machine-to-machine healthcare solution using Android mobile devices in Global networks, *IEEe Sens. J.* 13 (5) (2013) 1419–1424, <https://doi.org/10.1109/JSEN.2012.2236013>.
- [8] S. Ness, Integrating AI models for voltage and current monitoring in autonomous mobile robots to prevent power system blackouts, *IEEe Access* 13 (2025) 67079–67094, <https://doi.org/10.1109/ACCESS.2025.3560088>.
- [9] Á. Madridano, A. Al-Kaff, D. Martín, A. de la Escalera, Trajectory planning for multi-robot systems: methods and applications, *Expert. Syst. Appl.* 173 (2021) 114660, <https://doi.org/10.1016/j.eswa.2021.114660>.
- [10] J. Tan, S. Xue, Q. Guan, T. Niu, H. Cao, B. Chen, Unmanned aerial-ground vehicle finite-time docking control via pursuit-evasion games, *Nonlinear. Dyn.* 113 (13) (2025) 16757–16777, <https://doi.org/10.1007/s1071-025-11021-6>.
- [11] H. Bilal, M.S. Aslam, Y. Tian, I. Ullah, S. Ayouni, A.V. Vasilakos, A consumer electronics-enhanced UAV system for agricultural farm tracking with fuzzy SMO and actuator fault detection control algorithms, *IEEE Trans. Consum. Electron.* 71 (2) (2025) 6910–6923, <https://doi.org/10.1109/TCE.2025.3563993>.
- [12] J. Yang, T. Zhang, C.-Y. Tsai, Y. Lu, L. Yao, Evolution and emerging trends of named entity recognition: bibliometric analysis from 2000 to 2023, *Heliyon* (9) (2024) 10, <https://doi.org/10.1016/j.heliyon.2024.e30053>.
- [13] J. Alonso-Mora, R. Knepper, R. Siegwart, D. Rus, Local motion planning for collaborative multi-robot manipulation of deformable objects, *2015 IEEE Int. Conf. Robot. Autom. (ICRA)* (2015).
- [14] L. Emmi, E. Le Flécher, V. Cadenat, M. Devy, A hybrid representation of the environment to improve autonomous navigation of mobile robots in agriculture, *Precis. Agric.* 22 (2) (2021) 524–549, <https://doi.org/10.1007/s11119-020-09773-9>.
- [15] W. Ren, N. Sorensen, Distributed coordination architecture for multi-robot formation control, *Rob. Aut. Syst.* 56 (4) (2008) 324–333, <https://doi.org/10.1016/j.robot.2007.08.005>.
- [16] J. Alonso-Mora, E. Montijano, M. Schwager, D. Rus, Distributed multi-robot formation control among obstacles: A geometric and optimization approach with consensus, *2016 IEEE Int. Conf. Robot. Autom. (ICRA)* (2016).
- [17] H. Bilal, A. Rehman, M.S. Aslam, I. Ullah, W.J. Chang, N. Kumar, A. M. Almuhaideb, Hybrid TrafficAI: A generative AI framework for real-time traffic simulation and adaptive behavior modeling, *IEEE Trans. Intell. Transp. Syst.* (2025) 1–17, <https://doi.org/10.1109/TITS.2025.3571041>.
- [18] S. Cohen, N. Agmon, Recent advances in formations of multiple robots, *Curr. Robot. Rep.* 2 (2) (2021) 159–175, <https://doi.org/10.1007/s43154-021-00049-2>.
- [19] Y. Zhou, J. Xiao, Y. Zhou, G. Loianno, Multi-robot collaborative perception with graph Neural networks, *IEEE Robot. Autom. Lett.* 7 (2) (2022) 2289–2296, <https://doi.org/10.1109/LRA.2022.3141661>.
- [20] T. Balch, R.C. Arkin, Behavior-based formation control for multirobot teams, *IEEE Trans. Robot. Autom.* 14 (6) (1998) 926–939, <https://doi.org/10.1109/70.736776>.
- [21] J.P. Desai, J.P. Ostrowski, V. Kumar, Modeling and control of formations of nonholonomic mobile robots, *IEEE Trans. Robot. Autom.* 17 (6) (2001) 905–908, <https://doi.org/10.1109/70.976023>.
- [22] M.S. Aslam, H. Bilal, W.J. Chang, A. Yahya, I.A. Badruddin, S. Kamangar, M. Hussien, Formation control of heterogeneous multi-agent systems under fixed and switching hierarchies, *IEEe Access* 12 (2024) 97868–97882, <https://doi.org/10.1109/ACCESS.2024.3419815>.
- [23] J. Tan, S. Xue, H. Li, Z. Guo, H. Cao, B. Chen, Hierarchical safe reinforcement learning control for leader-follower systems with prescribed performance, *IEEE Trans. Autom. Sci. Eng.* 22 (2025) 19568–19581, <https://doi.org/10.1109/TASE.2025.3596912>.
- [24] M.A. Lewis, K.-H. Tan, High precision formation control of mobile robots using virtual structures, *Aut. Robots* 4 (4) (1997) 387–403, <https://doi.org/10.1023/A:1008814708459>.
- [25] W. Ren, Decentralization of virtual structures in formation control of multiple vehicle systems via consensus strategies, *Eur. J. Control* 14 (2) (2008) 93–103, <https://doi.org/10.3166/ejc.14.93-103>.
- [26] M.S. Aslam, H. Bilal, W.J. Chang, N. Kumar, I.A. Khan, A.V. Vasilakos, Hodelayed filtering of Markov jump fuzzy systems in consumer electronics: input-Output analysis, *IEEE Trans. Consum. Electron.* 71 (2) (2025) 7002–7013, <https://doi.org/10.1109/TCE.2025.3565105>.
- [27] S. Monteiro, E. Bicho, A dynamical systems approach to behavior-based formation control, in: *Proceedings 2002 IEEE International Conference on Robotics and Automation (Cat. No.02CH37292)*, 2002.
- [28] S. Xue, J. Liu, H. Cao, X. Zheng, H. Li, J. Zhang, Leader-following formation tracking of multiagent systems using adaptive scaling mechanism under spatial constraints, *IEEe Trans. Syst. Man Cybern.: Syst.* 54 (2) (2024) 1214–1225, <https://doi.org/10.1109/TSMC.2023.3323995>.
- [29] S. Sharmooz, M.S. Aslam, H. Liu, H. Bilal, A.V. Vasilakos, Modeling of asynchronous mode-dependent delays in stochastic markovian jumping modes based on static neural networks for robotic manipulators, *IEEE Trans. Autom. Sci. Eng.* 22 (2025) 13398–13410, <https://doi.org/10.1109/TASE.2025.3552645>.
- [30] Z. Sui, Z. Pu, J. Yi, S. Wu, Formation control with collision avoidance through deep reinforcement learning using model-guided demonstration, *IEEe Trans. Neural Netw. Learn. Syst.* 32 (6) (2021) 2358–2372, <https://doi.org/10.1109/TNNLS.2020.3004893>.
- [31] J. Tan, S. Xue, H. Li, Z. Guo, H. Cao, D. Li, Prescribed performance robust approximate optimal tracking control via Stackelberg Game, *IEEE Trans. Autom. Sci. Eng.* 22 (2025) 12871–12883, <https://doi.org/10.1109/TASE.2025.3549114>.
- [32] M.S. Aslam, A. Qamar, H. Bilal, A.V. Vasilakos, Actuator-Fault-Tolerant adaptive tracking control of delayed fuzzy systems using reinforcement learning, *IEEe Trans. Autom. Sci. Eng.* (2025) 1, <https://doi.org/10.1109/TASE.2025.3611686>.
- [33] C. Yan, X. Xiang, C. Wang, Fixed-wing UAVs flocking in continuous spaces: A deep reinforcement learning approach, *Rob. Aut. Syst.* 131 (2020) 103594, <https://doi.org/10.1016/j.robot.2020.103594>.
- [34] N. Xing, Q. Zong, B. Tian, Q. Wang, L. Dou, Nash network formation among unmanned aerial vehicles, *Wirel. Netw.* 26 (3) (2020) 1781–1793, <https://doi.org/10.1007/s11276-018-1866-1>.
- [35] H. Bilal, M.S. Obaidat, M.S. Aslam, J. Zhang, B. Yin, K. Mahmood, Online fault diagnosis of industrial robot using IoRT and hybrid deep learning techniques: an experimental approach, *IEEE Internet Things J.* 11 (19) (2024) 31422–31437, <https://doi.org/10.1109/JIOT.2024.3418352>.
- [36] J. Tan, S. Xue, Q. Guan, K. Qu, H. Cao, Finite-time safe reinforcement learning control of multi-player nonzero-sum game for quadcopter systems, *Inf. Sci. (Ny)* 712 (2025) 122117, <https://doi.org/10.1016/j.ins.2025.122117>.

- [37] E.G. Hernandez-Martinez, J.M.F. Albino, Hybrid architecture of multi-robot systems based on formation control and SOM neural networks, in: 2011 IEEE International Conference on Control Applications (CCA), 2011.
- [38] A.M. Hasan, S.M. Rafaat, Optimized formation control of multi-agent system using PSO algorithm, *Indones. J. Electr. Eng. Comput. Sci.* 20 (3) (2020) 1591.
- [39] M.A. Kamel, X. Yu, Y. Zhang, Real-time fault-tolerant formation control of multiple WMRs based on hybrid GA-PSO algorithm, *IEEE Trans. Autom. Sci. Eng.* 18 (3) (2021) 1263–1276, <https://doi.org/10.1109/TASE.2020.3000507>.
- [40] A. Noormohammadi Asl, M.B. Menhaj, A. Sajedin, Control of leader–follower formation and path planning of mobile robots using Asexual Reproduction Optimization (ARO), *Appl. Soft. Comput.* 14 (2014) 563–576, <https://doi.org/10.1016/j.asoc.2013.07.030>.
- [41] H. Qiu, H. Duan, Receding horizon control for multiple UAV formation flight based on modified brain storm optimization, *Nonlinear. Dyn.* 78 (3) (2014) 1973–1988, <https://doi.org/10.1007/s11071-014-1579-7>.
- [42] B. Zhang, H. Duan, Three-dimensional path planning for uninhabited combat aerial vehicle based on predator-prey pigeon-inspired optimization in dynamic environment, *IEEE/ACM Trans. Comput. Biol. Bioinform.* 14 (1) (2017) 97–107, <https://doi.org/10.1109/TCBB.2015.2443789>.
- [43] H. Duan, P. Qiao, Pigeon-inspired optimization: a new swarm intelligence optimizer for air robot path planning, *Int. J. Intell. Comput. Cybern.* 7 (1) (2014) 24–37, <https://doi.org/10.1108/IJICC-02-2014-0005>.
- [44] X.-J. Wu, L. Xu, R. Zhen, X.-L. Wu, Global and local moth-flame optimization algorithm for UAV formation path planning under multi-constraints, *Int. J. Control Autom. Syst.* 21 (3) (2023) 1032–1047, <https://doi.org/10.1007/s12555-020-0979-3>.
- [45] G. Li, T. Zhang, C.-Y. Tsai, L. Yao, Y. Lu, J. Tang, Review of the metaheuristic algorithms in applications: visual analysis based on bibliometrics, *Expert. Syst. Appl.* 255 (2024) 124857, <https://doi.org/10.1016/j.eswa.2024.124857>.
- [46] Q. Liu, M. Wang, A projection-based algorithm for optimal formation and optimal matching of multi-robot system, *Nonlinear. Dyn.* 104 (1) (2021) 439–450, <https://doi.org/10.1007/s11071-020-06189-y>.
- [47] G. Li, T. Zhang, C.-Y. Tsai, Y. Lu, J. Yang, L. Yao, Modified crayfish optimization algorithm with adaptive spiral elite greedy opposition-based learning and search-hide strategy for global optimization, *J. Comput. Des. Eng.* 11 (4) (2024) 249–305, <https://doi.org/10.1093/jcde/qwae069>.
- [48] H. Jia, H. Rao, C. Wen, S. Mirjalili, Crayfish optimization algorithm, *Artif. Intell. Rev.* 56 (2) (2023) 1919–1979, <https://doi.org/10.1007/s10462-023-10567-4>.
- [49] F. Marini, B. Walczak, Particle swarm optimization (PSO). A tutorial, *Chemom. Intell. Lab. Syst.* 149 (2015) 153–165, <https://doi.org/10.1016/j.chemolab.2015.08.020>.
- [50] A.A. Heidari, S. Mirjalili, H. Faris, I. Aljarah, M. Mafarja, H. Chen, Harris hawks optimization: algorithm and applications, *Future Gener. Comput. Syst.* 97 (2019) 849–872, <https://doi.org/10.1016/j.future.2019.02.028>.
- [51] S. Mirjalili, SCA: A Sine Cosine algorithm for solving optimization problems, *Knowl. Based. Syst.* 96 (2016) 120–133, <https://doi.org/10.1016/j.knosys.2015.12.022>.
- [52] M. Brand, M. Masuda, N. Wehner, Y. Xiao-Hua, Ant Colony optimization algorithm for robot path planning, in: 2010 International Conference On Computer Design and Applications, 2010.
- [53] Q. Chen, L. Yao, T. Zhang, Y. Lu, Z. Han, H. Zhao, Multi-strategy improved Parrot optimization for feature engineering in machine learning, *Clust. Comput* 28 (13) (2025) 857, <https://doi.org/10.1007/s10586-025-05613-5>.
- [54] R.M. Devi, M. Premkumar, G. Kiruthiga, R. Sowmya, IGJO: an improved golden jackel optimization algorithm using local escaping operator for feature selection problems, *Neural Process. Lett.* 55 (5) (2023) 6443–6531, <https://doi.org/10.1007/s11063-023-11146-y>.
- [55] X. Chen, M. Zhang, M. Yang, D. Wang, A multi-strategy improved beluga whale optimization algorithm for constrained engineering problems, *Clust. Comput* 27 (10) (2024) 14685–14727, <https://doi.org/10.1007/s10586-024-04680-4>.
- [56] X. Meng, J. Jiang, H. Wang, AGWO: advanced GWO in multi-layer perception optimization, *Expert. Syst. Appl.* 173 (2021) 114676, <https://doi.org/10.1016/j.eswa.2021.114676>.
- [57] J. Han, X. Wang, L. Yan, A. Dahlak, Modelling the performance of an SOEC by optimization of neural network with MPSO algorithm, *Int. J. Hydrog. Energy* 44 (51) (2019) 27947–27957, <https://doi.org/10.1016/j.ijhydene.2019.09.055>.
- [58] A.R. Kazemi, S. Tohidi, M.R. Mosavi, Enhancing classification performance between different GNSS interferences using neural networks trained by TAC-PSO algorithm, in: 2020 10th International Symposium on Telecommunications (IST), 2020.
- [59] C. De La Cruz, R. Carelli, Dynamic model based formation control and obstacle avoidance of multi-robot systems, *Robotica* 26 (3) (2008) 345–356, <https://doi.org/10.1017/S0263574707004092>.
- [60] L. Yao, P. Yuan, C.-Y. Tsai, T. Zhang, Y. Lu, S. Ding, ESO: an enhanced snake optimizer for real-world engineering problems, *Expert. Syst. Appl.* 230 (2023) 120594, <https://doi.org/10.1016/j.eswa.2023.120594>.
- [61] A.B. Gabis, Y. Meraihi, S. Mirjalili, A. Ramdane-Cherif, A comprehensive survey of sine cosine algorithm: variants and applications, *Artif. Intell. Rev.* 54 (7) (2021) 5469–5540, <https://doi.org/10.1007/s10462-021-10026-y>.
- [62] G. Sun, Y. Lan, R. Zhao, Differential evolution with Gaussian mutation and dynamic parameter adjustment, *Soft. Comput.* 23 (5) (2019) 1615–1642, <https://doi.org/10.1007/s00500-017-2885-z>.
- [63] L. Yao, G. Li, P. Yuan, J. Yang, D. Tian, T. Zhang, Reptile search algorithm considering different flight heights to solve engineering optimization design problems, *Biomimetics* 8 (3) (2023) 305, <https://www.mdpi.com/2313-7673/8/3/305>.
- [64] S. Cheng, Y. Shi, Q. Qin, T.O. Ting, R. Bai, Maintaining population diversity in brain storm optimization algorithm, in: 2014 IEEE Congress on Evolutionary Computation (CEC), 2014.
- [65] L. Yao, J. Yang, P. Yuan, G. Li, Y. Lu, T. Zhang, Multi-strategy improved sand cat swarm optimization: global optimization and feature selection, *Biomimetics* 8 (6) (2023).
- [66] Y. Li, C. Wu, W. Tian, Formation outlier Formation transformation based on virtual robots, in: 2023 China Automation Congress (CAC), 2023.
- [67] M.S. Aslam, H. Bilal, A.V. Vasilakos, Self-triggered scheme design for Takagi-Sugeno fuzzy model based on mismatch premise variable with time-varying delay, *IEEE Trans. Autom. Sci. Eng.* 22 (2025) 15536–15548, <https://doi.org/10.1109/TASE.2025.3570089>.
- [68] X. Yuan, L. Yao, T. Zhang, Q. Chen, G. Li, An effective method for solving feature selection problems based on multi-strategy Hippo optimization algorithm, *J. Comput. Des. Eng.* 12 (8) (2025) 193–251, <https://doi.org/10.1093/jcde/qwaf073>.